Application of AI in Detecting and Preventing Brute-Force Attacks on SSH Systems with Early Prediction

FPT University - Information Assurance

2026

Contents

1 FRONT MATTER 4
1.1 ABSTRACT . . . 4
1.2 ACKNOWLEDGEMENT . . . 5
1.3 LIST OF FIGURES . . . 5
1.4 LIST OF TABLES . . . 6
1.5 ABBREVIATIONS . . . 7

2 CHAPTER 1: INTRODUCTION 9
2.1 1.1 Background . . . 9
2.2 1.2 Problem Statement . . . 11
2.3 1.3 Research Objectives . . . 12
2.4 1.4 Significance of the Study . . . 13
2.4.1 1.4.1 Scientific Significance . . . 13
2.4.2 1.4.2 Practical Significance . . . 14
2.5 1.5 Scope and Limitations . . . 14
2.5.1 1.5.1 Scope . . . 14
2.5.2 1.5.2 Limitations . . . 15
2.6 1.6 Thesis Structure . . . 15
2.7 References for Chapter 1 . . . 16

3 CHAPTER 2: LITERATURE REVIEW 18
3.1 2.1 Review of Previous Studies . . . 18
3.1.1 2.1.1 The SSH Protocol and Authentication Mechanisms . . . 18
3.1.2 2.1.2 Classification of Brute-Force Attacks . . . 19
3.1.3 2.1.3 Traditional Detection Methods . . . 20
3.1.4 2.1.4 Machine Learning in Intrusion Detection . . . 21
3.1.5 2.1.5 Anomaly Detection Algorithms . . . 22
3.1.6 2.1.6 Dynamic Thresholding in Anomaly Detection . . . 23
3.1.7 2.1.7 The ELK Stack for Security Monitoring . . . 24
3.1.8 2.1.8 International Research . . . 25
3.1.9 2.1.9 Domestic Research (Vietnam) . . . 26
3.1.10 2.1.10 Summary of Research Landscape . . . 27
3.2 2.2 Summary of the Literature Review . . . 28
3.3 2.3 Contribution of Research . . . 29
3.4 References for Chapter 2 . . . 30

4 CHAPTER 3: METHODOLOGY 35
4.1 3.1 Research Design . . . 35
4.1.1 3.1.1 Overall System Architecture . . . 35
4.1.2 3.1.2 Semi-Supervised Anomaly Detection Approach . . . 36
4.2 3.2 Data Collection Methods . . . 37
4.2.1 3.2.1 Data Sources . . . 37

4.2.2 3.2.2 Log Data Format . . . 38
4.2.3 3.2.3 Labeling Strategy . . . 38
4.2.4 3.2.4 Data Splitting . . . 38
4.3 3.3 Sampling Data Analysis Techniques . . . 39
4.3.1 3.3.1 Feature Extraction via Sliding Window . . . 39
4.3.2 3.3.2 Description of the 14 Features . . . 39
4.3.3 3.3.3 Temporal Feature Engineering Rationale . . . 41
4.3.4 3.3.4 Feature Scaling . . . 42
4.3.5 3.3.4 Model Selection and Configuration . . . 43
4.3.6 3.3.5 Dynamic Threshold Algorithm . . . 43
4.3.7 3.3.6 Training Process . . . 45
4.4 3.4 Limitations of the Methodology . . . 46
4.4.1 3.4.1 Real-Time Detection Pipeline . . . 46
4.4.2 3.4.2 Fail2Ban Integration and Alert System . . . 47
4.4.3 3.4.3 Evaluation Metrics . . . 47
4.4.4 3.4.4 Experimental Design for Attack Scenario Evaluation . . . 48
4.4.5 3.4.5 Ethical Considerations . . . 49
4.4.6 3.4.6 Methodological Constraints . . . 49

5 CHAPTER 4: EXPERIMENTAL AND RESULTS 51
5.1 4.1 Introduction . . . 51
5.2 4.2 Presentation of Data . . . 52
5.2.1 4.2.1 Overall Dataset Statistics . . . 52
5.2.2 4.2.2 Honeypot Data Analysis . . . 52
5.2.3 4.2.3 Simulation Data Analysis . . . 53
5.2.4 4.2.4 Feature Comparison Between Normal and Attack Classes . . . 54
5.2.5 4.2.5 Statistical Summary of the Combined Dataset . . . 54
5.3 4.3 Analysis of Results . . . 55
5.3.1 4.3.1 Feature Distribution Analysis . . . 55
5.3.2 4.3.2 Correlation Analysis . . . 55
5.3.3 4.3.3 Feature Importance Analysis . . . 56
5.3.4 4.3.4 Baseline Model Performance . . . 57
5.3.5 4.3.5 Detailed Model Analysis . . . 57
5.3.6 4.3.6 Summary of Baseline Analysis . . . 58
5.4 4.4 Interpretation of Results . . . 59
5.4.1 4.4.1 Optimized Model Performance . . . 59
5.4.2 4.4.2 Justification for Selecting Isolation Forest . . . 59
5.4.3 4.4.3 Dynamic Threshold Effectiveness . . . 60
5.4.4 4.4.4 Parameter Sensitivity Analysis . . . 60
5.4.5 4.4.5 Optimization Process Details . . . 61
5.5 4.5 Comparison with Literature . . . 62
5.6 4.6 Implications of the Results . . . 62
5.6.1 4.6.1 Attack Scenario Testing . . . 62
5.6.2 4.6.2 Results by Scenario . . . 63
5.6.3 4.6.3 Detailed Scenario 3 Analysis: Low-and-Slow . . . 64
5.6.4 4.6.4 Consolidated Scenario Analysis . . . 65
5.6.5 4.6.4 Real-Time System Performance . . . 66
5.6.6 4.6.5 Summary of Experimental Findings . . . 67

6 CHAPTER 5: DISCUSSION 69
6.1 5.1 Restate the Research Problem or Objectives . . . 69
6.2 5.2 Summarize Key Findings . . . 70
6.2.1 5.2.1 Overall Model Performance Assessment . . . 70
6.2.2 5.2.2 The Primacy of Temporal Features . . . 71
6.2.3 5.2.2 The Value of Dynamic Thresholding for Early Prediction . . . 71

6.2.4 5.2.3 The Trade-Off Between False Positive Rate and Security 72
6.2.5 5.2.4 Comparative Positioning in the Literature 73
6.2.6 5.2.5 Practical Implications for Security Operations 73
6.2.7 5.2.6 System Scalability and Deployment Considerations 74
6.2.8 5.2.6 Attack Scenario Implications 74
6.2.9 5.2.7 Limitations of the Research 75

7 CHAPTER 6: CONCLUSION AND FUTURE WORK 78
7.1 6.1 Conclusion 78
7.1.1 Contribution 1: Semi-Supervised Anomaly Detection System Based on Isolation Forest 78
7.1.2 Contribution 2: EWMA-Adaptive Percentile Dynamic Threshold Algorithm 79
7.1.3 Contribution 3: Comprehensive 14-Feature Behavioral Feature Set 79
7.1.4 Contribution 4: End-to-End Integrated System 79
7.1.5 Contribution 5: Comprehensive Behavioral Feature Design and Validation 80
7.1.6 Contribution 6: Evaluation on Real-World Data with Practical Attack Scenarios 80
7.1.7 Achievement of Research Objectives 80
7.1.8 Reflection on Research Methodology 81
7.2 6.2 Future Work 81
7.2.1 6.2.1 Application of Deep Learning 81
7.2.2 6.2.2 Online Learning and Continuous Adaptation 82
7.2.3 6.2.3 Multi-Protocol Support 82
7.2.4 6.2.4 Federated Detection 82
7.2.5 6.2.5 Threat Intelligence Integration 82
7.2.6 6.2.6 SOAR Integration 82
7.2.7 6.2.7 Ensemble and Hybrid Detection Methods 83
7.2.8 6.2.8 Explainability and Interpretability 83
7.2.9 6.2.9 Long-Term IP Profiling 83
7.2.10 6.2.10 Cross-Environment Transfer Learning 83
7.2.11 6.2.11 Real-World Deployment Validation 84
7.3 REFERENCES 84
7.4 APPENDICES 88
7.4.1 Appendix A: Feature Extraction Code Snippets 88
7.4.2 Appendix B: Docker Compose Configuration 88
7.4.3 Appendix C: Isolation Forest Hyperparameter Tuning Results 88
7.4.4 Appendix D: Attack Simulation Scripts 88
7.4.5 Appendix E: Logstash Pipeline Configuration 88
7.4.6 Appendix F: Dynamic Threshold Parameter Sensitivity Analysis 88

Chapter 1

FRONT MATTER

1.1 ABSTRACT

The rapid growth of digital infrastructure has made cybersecurity an increasingly critical concern for organizations worldwide. Among the most persistent and prevalent threats targeting server infrastructure is the brute-force attack on the Secure Shell (SSH) protocol, which remains one of the most widely used mechanisms for remote system administration. Conventional defense tools such as Fail2Ban rely on static thresholds to detect and block malicious login attempts; however, these approaches are inherently limited in their ability to identify sophisticated attack variants, including slow brute-force, distributed brute-force, and credential stuffing attacks. This thesis presents the design, implementation, and evaluation of an intelligent SSH brute-force attack detection and prevention system that leverages unsupervised machine learning, specifically the Isolation Forest algorithm, combined with a novel EWMA-Adaptive Percentile dynamic thresholding mechanism to enable early prediction of attacks before they escalate to full intensity.

The proposed system was evaluated using a hybrid dataset comprising real-world attack data collected from an SSH honeypot (119,729 log lines from 679 unique IP addresses) and simulated normal behavior data (54,521 log lines from 64 users). A set of 14 behavioral features was engineered and extracted over 5-minute sliding windows per source IP address, capturing temporal, authentication, connection, and session characteristics. Three unsupervised anomaly detection algorithms were systematically compared: Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class Support Vector Machine (OCSVM). After optimization, the Isolation Forest model achieved an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. The OCSVM model achieved the highest accuracy of 91.38% and F1-score of 94.55%, while LOF achieved a perfect recall of 100% but exhibited the highest false positive rate of 67.10%.

The system was deployed as a complete end-to-end solution using a Docker-based microservices architecture comprising 9 services, integrating FastAPI, React, the ELK Stack (Elasticsearch, Logstash, Kibana),

and Fail2Ban for automated response. The EWMA-Adaptive Percentile dynamic threshold (alpha=0.3, base_percentile=95, sensitivity_factor=1.5) enables the system to adapt to evolving traffic patterns and provides early warning capabilities, detecting low-and-slow attacks within 3-5 attempts (approximately 2-3 minutes), significantly outperforming traditional static threshold methods. Five attack scenarios were designed and tested, demonstrating robust detection across basic brute-force, distributed, credential stuffing, and dictionary attacks, with the low-and-slow scenario representing the most challenging case.

Keywords: SSH brute-force detection, Isolation Forest, anomaly detection, dynamic threshold, EWMA, unsupervised machine learning, ELK Stack, intrusion detection system, cybersecurity, early prediction

1.2 ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all those who have supported and guided me throughout the completion of this thesis.

First and foremost, I extend my sincere thanks to my thesis advisor for their invaluable guidance, constructive feedback, and unwavering encouragement throughout the entire research process. Their expertise in cybersecurity and machine learning has been instrumental in shaping the direction and quality of this work.

I am profoundly grateful to the faculty members of the Information Assurance program at FPT University for providing me with a solid foundation in cybersecurity principles, programming, and research methodology. The knowledge and skills I acquired during my studies were essential to the successful completion of this project.

I would also like to thank the members of my thesis committee for their time, insightful comments, and valuable suggestions, which have helped improve the quality and rigor of this research.

Special thanks go to my classmates and colleagues who provided technical assistance, shared their experiences, and offered moral support during challenging phases of the project. The collaborative and stimulating academic environment at FPT University has been a constant source of motivation.

I am grateful to the open-source community for developing and maintaining the tools and frameworks that formed the backbone of this research, including scikit-learn, FastAPI, the Elastic Stack, Docker, and Fail2Ban. Without these freely available and well-documented tools, this work would not have been possible.

Finally, I owe my deepest gratitude to my family for their unconditional love, patience, and support throughout my academic journey. Their belief in my abilities has been a constant source of strength and inspiration.

1.3 LIST OF FIGURES

- Figure 1.1: Global trends in SSH brute-force attacks (2018-2025)
- Figure 1.2: Comparison of traditional vs. AI-based attack detection approaches

- Figure 1.3: Overview of the proposed system architecture (block diagram)
- Figure 2.1: Layered architecture of the SSH-2 protocol
- Figure 2.2: Illustration of static threshold vs. dynamic threshold (EWMA, Adaptive Percentile, and hybrid)
- Figure 2.3: Architecture for integrating AI models with the ELK Stack for SSH monitoring
- Figure 2.4: Venn diagram showing the research positioning at the intersection of three domains
- Figure 3.1: Overall system architecture for SSH brute-force attack detection
- Figure 3.2: Data flow pipeline of the system
- Figure 3.3: Illustration of the sliding window method (window=5 min, stride=1 min)
- Figure 3.4: Model training and evaluation workflow
- Figure 3.5: Illustration of the EWMA-Adaptive Percentile dynamic threshold algorithm
- Figure 3.6: Five-stage real-time detection pipeline
- Figure 4.1: Distribution of failed authentication attempts per IP (honeypot)
- Figure 4.2: Comparison of hourly activity distribution between honeypot (attack) and simulation (normal)
- Figure 4.3: Boxplot comparison of key feature distributions between Normal and Attack classes
- Figure 4.4: Histogram distribution of 14 features, separated by normal/attack labels
- Figure 4.5: Pearson correlation matrix heatmap of 14 features
- Figure 4.6: Bar chart of feature importance for 14 features
- Figure 4.7: Anomaly score distribution on the test set for three models
- Figure 4.8: Violin plot comparing anomaly score distributions between normal and attack for three models
- Figure 4.9: Radar chart comparing five performance metrics across three models
- Figure 4.10: ROC curves for three models on the test set
- Figure 4.11: Precision-Recall curves for three models
- Figure 4.12: Anomaly score and dynamic threshold evolution over time on test data
- Figure 4.13: Effect of alpha, base_percentile, and sensitivity_factor on F1-Score
- Figure 4.14: Heatmap of detection rates by attack scenario and model

1.4 LIST OF TABLES

- Table 1.1: Summary of research challenges and proposed solutions
- Table 1.2: Research objectives and corresponding approaches
- Table 2.1: Comparison of SSH brute-force attack variants
- Table 2.2: Comparison of advantages and disadvantages of traditional detection methods
- Table 2.3: Comparison of Isolation Forest, LOF, and One-Class SVM characteristics
- Table 2.4: Comparative summary of related research works
- Table 3.1: List of 9 Docker services in the system
- Table 3.2: Summary of two data sources

- Table 3.3: Dataset split summary
- Table 3.4: Summary of 14 features and their significance
- Table 3.5: Comparison of characteristics of three models
- Table 3.6: Dynamic threshold algorithm parameters and their meanings
- Table 3.7: Summary of evaluation metrics and their security significance
- Table 4.1: Overall dataset statistics
- Table 4.2: Detailed honeypot data statistics
- Table 4.3: Detailed simulation data statistics
- Table 4.4: Descriptive statistics of key features by label
- Table 4.5: Feature importance ranking (Top 5)
- Table 4.6: Performance comparison of three models on the test set (baseline)
- Table 4.7: Confusion matrix estimates for three models
- Table 4.8: Comparison with related studies
- Table 4.9: Dynamic threshold vs. static threshold comparison
- Table 4.10: Description of 5 attack scenarios
- Table 4.11: Low-and-Slow detection results by model
- Table 4.12: Summary of detection results across 5 attack scenarios
- Table 4.13: Test environment configuration
- Table 4.14: Latency analysis by pipeline stage
- Table 4.15: Processing throughput by model
- Table 4.16: Resource utilization of main services
- Table 5.1: Optimized model performance comparison
- Table 5.2: Comparison with related works

1.5 ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AUC	Area Under the Curve
API	Application Programming Interface
BST	Binary Search Tree
CPU	Central Processing Unit
DNS	Domain Name System
ELK	Elasticsearch, Logstash, Kibana
EWMA	Exponentially Weighted Moving Average
FN	False Negative

Abbreviation	Full Form
FP	False Positive
FPR	False Positive Rate
FTP	File Transfer Protocol
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
IF	Isolation Forest
IETF	Internet Engineering Task Force
IoT	Internet of Things
IP	Internet Protocol
IQR	Interquartile Range
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
ML	Machine Learning
MTTR	Mean Time to Respond
NCSC	National Cyber Security Center
NIDS	Network Intrusion Detection System
OCSVM	One-Class Support Vector Machine
PAM	Pluggable Authentication Module
RAM	Random Access Memory
RBF	Radial Basis Function
RDP	Remote Desktop Protocol
RFC	Request for Comments
ROC	Receiver Operating Characteristic
SFTP	SSH File Transfer Protocol
SCP	Secure Copy Protocol
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SOAR	Security Orchestration, Automation and Response
SSH	Secure Shell
SVM	Support Vector Machine
TCP	Transmission Control Protocol
TN	True Negative
TP	True Positive
TPR	True Positive Rate

Chapter 2

CHAPTER 1: INTRODUCTION

2.1 1.1 Background

In the context of accelerating digital transformation across the globe, information technology infrastructure plays an increasingly critical role in the operations of organizations, enterprises, and government agencies. According to a report by Cybersecurity Ventures, the global cost of cybercrime is projected to reach 10.5 trillion USD annually by 2025, a dramatic increase from 3 trillion USD in 2015 [1]. This rapid escalation reflects the growing complexity and scale of cyberattacks, underscoring the urgent need for advanced and effective cybersecurity solutions.

The Secure Shell (SSH) protocol is among the most widely used protocols for remote system administration, secure file transfer, and establishing encrypted communication channels. Originally developed in 1995 by Tatu Ylonen at the Helsinki University of Technology, SSH has become the industry standard for managing Linux/Unix servers [2]. However, its very ubiquity has made SSH a primary target for cyberattacks, particularly brute-force attacks, in which an adversary systematically attempts thousands to millions of username-password combinations in order to gain unauthorized access to a system.

Data from the SANS Internet Storm Center indicate that TCP port 22 (SSH) consistently ranks among the top five most scanned and attacked ports on the Internet [3]. A 2023 report by Rapid7 found that, on average, a publicly accessible SSH server receives more than 10,000 unauthorized login attempts per day [4]. In Vietnam, the National Cyber Security Center (NCSC) has reported that SSH brute-force attacks constitute a significant proportion of total cybersecurity incidents recorded annually, with a steadily increasing trend from 2020 onward [5].

Traditional methods of defending against SSH brute-force attacks include rate limiting (restricting the number of failed login attempts), IP blacklisting, public key authentication, and deploying tools such as Fail2Ban or DenyHosts [6]. While these approaches have demonstrated a measure of effectiveness, they suffer from several

well-documented limitations. First, methods based on static thresholds are easily circumvented by attackers who reduce the speed of their attempts (slow brute-force) or distribute the attack across multiple IP addresses (distributed brute-force) [7]. Second, the lack of learning and adaptive capabilities prevents traditional systems from recognizing novel attack variants. Third, high false positive rates can disrupt service for legitimate users by erroneously blocking their access.

Against this backdrop, artificial intelligence (AI) and machine learning (ML) have emerged as a highly promising approach to the problem of detecting and preventing cyberattacks. Machine learning models possess the capability to analyze large volumes of log data, identify anomalous behavioral patterns, and issue alerts in real time — capabilities that rule-based static methods are inherently unable to match [8]. In particular, unsupervised anomaly detection algorithms such as Isolation Forest, Local Outlier Factor (LOF), and One-Class SVM offer a distinct advantage in detecting previously unknown forms of attack, because they do not require labeled data for training [9].

This research was motivated by the practical need to develop an intelligent SSH brute-force attack detection and prevention system with early prediction capabilities, enabling the system to identify attacks before they inflict damage. The proposed system integrates the Isolation Forest algorithm with an EWMA-Adaptive Percentile dynamic thresholding mechanism, deployed on an ELK Stack (Elasticsearch, Logstash, Kibana) and Fail2Ban infrastructure, to create a comprehensive solution encompassing data collection, analysis, detection, and automated response.

The growing reliance on cloud computing and Infrastructure-as-a-Service (IaaS) models has further amplified the importance of SSH security. Major cloud providers including Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure all use SSH as the default protocol for remote access to virtual machine instances. According to Shodan, the specialized search engine for Internet-connected devices, there are over 20 million SSH servers publicly accessible on the Internet at any given time. Each of these servers represents a potential target for brute-force attacks, and the automation of such attacks through botnets and commercial attack tools has made the threat landscape increasingly hostile.

The financial impact of successful SSH compromises extends far beyond the immediate cost of the intrusion itself. Once an attacker gains SSH access to a server, the compromise typically leads to lateral movement within the organization's network, data exfiltration, ransomware deployment, or the installation of cryptocurrency miners. IBM's 2023 Cost of a Data Breach report estimates the average cost of a data breach at 4.45 million USD, with compromised credentials being the most common initial attack vector, responsible for 15% of all breaches. In the Vietnamese context, the Ministry of Information and Communications has emphasized the need for AI-driven cybersecurity solutions as part of the national digital transformation strategy for the period 2021-2025, creating both an institutional imperative and a research opportunity for the development of intelligent security systems.

The convergence of these factors — the ubiquity of SSH, the increasing sophistication of brute-force attacks,

the limitations of traditional detection methods, and the maturation of machine learning techniques — creates a compelling case for the research presented in this thesis. By combining the Isolation Forest algorithm with a novel dynamic thresholding mechanism and deploying the solution on a modern ELK Stack infrastructure, this research aims to advance the state of the art in SSH security while providing a practical, deployable system for real-world use.

2.2 1.2 Problem Statement

SSH brute-force attacks represent one of the most persistent and prevalent forms of cyberattack. Despite the existence of numerous deployed countermeasures, the problem of detecting and preventing such attacks remains beset by several unresolved challenges.

Challenge 1: The evolution of attack techniques. Modern attackers no longer rely on simple, high-speed brute-force methods. Instead, they employ a range of sophisticated evasion techniques, including: slow brute-force attacks, in which the interval between attempts is deliberately lengthened to evade rate-limiting mechanisms; distributed brute-force attacks, which leverage botnets comprising thousands of IP addresses; intelligent dictionary attacks, employing customized password lists tailored to the target; and credential stuffing attacks, which exploit login credentials leaked from prior data breaches [10].

Challenge 2: Limitations of static thresholds. Traditional tools such as Fail2Ban operate on the basis of fixed thresholds — for example, blocking an IP address after 5 failed login attempts within a 10-minute window. This approach creates two opposing problems: if the threshold is set too low, legitimate users who mistype their password may be erroneously blocked (false positives); if the threshold is set too high, attackers can conduct slow brute-force campaigns without being detected (false negatives) [11]. Moreover, static thresholds are unable to adapt to the natural fluctuation of network traffic over time — peak hours and off-hours exhibit fundamentally different behavioral patterns.

Challenge 3: Lack of early prediction capability. The majority of existing solutions operate in a reactive mode, responding only after an attack has already occurred, rather than proactively predicting an attack before it escalates. The reconnaissance and initial probing phases — during which an attacker tests a small number of credential combinations to evaluate the target — are typically not identified, thereby forfeiting the opportunity to intervene before a full-scale attack is launched [12].

Challenge 4: Integration and automation. Many studies on the application of AI to attack detection focus exclusively on algorithmic aspects without addressing the practical problem of integration into operational infrastructure. The gap between research prototypes and deployable production systems (the research-to-deployment gap) remains a significant barrier to the practical adoption of AI in cybersecurity [13].

In light of these challenges, the primary research question of this thesis is formulated as follows:

How can an AI-based SSH brute-force attack detection and prevention system be constructed that is

capable of early attack prediction, dynamically adapts to changes in traffic patterns, and integrates fully into modern security monitoring infrastructure?

The subsidiary research questions are:

1. How effectively does the Isolation Forest algorithm detect SSH brute-force attacks compared to other unsupervised anomaly detection algorithms (LOF, One-Class SVM)?
2. To what extent does the EWMA-Adaptive Percentile dynamic thresholding mechanism improve detection performance relative to traditional static thresholds?
3. What degree of early prediction does the proposed system achieve across different brute-force attack scenarios?
4. Can the integrated architecture combining ELK Stack, Isolation Forest, and Fail2Ban meet the requirements for real-time monitoring and automated response in a practical environment?

2.3 1.3 Research Objectives

This research aims to design, implement, and evaluate an AI-based SSH brute-force attack detection and prevention system that integrates early prediction and automated response capabilities, operating on an ELK Stack platform with an adaptive dynamic thresholding mechanism. The research is guided by a clear set of goals that address the identified challenges systematically, spanning the full lifecycle from data collection and feature engineering through model training and evaluation to system deployment and real-time operation.

The specific objectives are as follows:

Objective 1: Design a comprehensive SSH behavioral feature set. Design and implement a set of 14 features extracted from SSH log data over 5-minute time windows per source IP address, encompassing features related to frequency, temporal distribution, authentication patterns, and connection behavior. This feature set must comprehensively capture the behavioral aspects necessary to distinguish between normal activity and attacks.

Objective 2: Train and evaluate anomaly detection models. Implement and systematically compare the performance of three unsupervised anomaly detection algorithms: Isolation Forest, Local Outlier Factor (LOF), and One-Class SVM. The target performance is a minimum F1-score of 85% and a minimum recall of 95% to ensure a high attack detection rate with minimal missed detections.

Objective 3: Develop an adaptive dynamic thresholding mechanism. Design and implement a dynamic threshold method combining EWMA (Exponentially Weighted Moving Average) and Adaptive Percentile, enabling the system to automatically adjust its detection threshold in response to variations in network traffic, thereby minimizing false alarm rates while maintaining high detection sensitivity.

Objective 4: Build a complete integrated system. Design a system architecture integrating the ELK Stack (Elasticsearch, Logstash, Kibana) for data collection and visualization, the Isolation Forest model for anomaly

detection, and Fail2Ban for automated response. The entire system is containerized using Docker to ensure portability and reproducibility.

Objective 5: Evaluate the system against realistic attack scenarios. Design and execute 5 simulated SSH brute-force attack scenarios with varying characteristics — basic brute-force, slow brute-force, distributed attack, dictionary attack, and credential stuffing — to comprehensively evaluate the detection and early prediction capabilities of the system. Each scenario is designed to test a specific aspect of the system's capabilities: the basic and dictionary scenarios test detection of straightforward attacks, the distributed and credential stuffing scenarios test detection of attacks that evade per-IP thresholds, and the slow brute-force scenario tests the early prediction capabilities of the dynamic threshold mechanism.

These five objectives collectively address the full spectrum of challenges identified in the problem statement: the evolution of attack techniques (Objectives 1, 5), the limitations of static thresholds (Objective 3), the lack of early prediction (Objectives 3, 5), and the integration and automation gap (Objective 4). The measurable targets (F1>85%, Recall>95%) provide concrete criteria for evaluating the success of the research.

2.4 1.4 Significance of the Study

2.4.1 1.4.1 Scientific Significance

This research contributes to the fields of cybersecurity and artificial intelligence on several fronts. First, the thesis provides a systematic comparative analysis of three unsupervised anomaly detection algorithms — Isolation Forest, LOF, and One-Class SVM — in the specific context of SSH brute-force attack detection. Although these algorithms have been extensively studied for general network intrusion detection, their evaluation on real-world SSH attack data from a honeypot remains limited in the existing literature.

Second, the study proposes the EWMA-Adaptive Percentile dynamic thresholding method, a novel approach compared to the fixed thresholds or simple dynamic thresholds commonly used in the literature. This mechanism enables the system to adapt to the natural variability of network traffic while maintaining high sensitivity to attack patterns.

Third, the set of 14 SSH behavioral features designed in this study can serve as a reference foundation for future research on SSH-based anomaly detection, particularly in terms of representing the differences between normal behavior and attacks within short time windows (5 minutes).

Fourth, the research demonstrates that temporal features — specifically session duration and inter-attempt timing — are the most discriminative characteristics for distinguishing automated brute-force attacks from legitimate SSH activity, a finding with implications for the design of future detection systems. Feature importance analysis revealed that session_duration_mean (5.50%), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%) are the top three most important features, outweighing traditional count-based features such as fail_count.

2.4.2 1.4.2 Practical Significance

From a practical perspective, the system developed in this research can be deployed directly into the operational environments of organizations and enterprises. The Docker-based architecture ensures portability and rapid deployment. The use of the ELK Stack — a widely adopted open-source toolset — reduces deployment costs compared to commercial solutions.

The system provides real-time visual monitoring through the Kibana dashboard and a custom React-based interface, enabling security administrators to track SSH security status in real time. Integration with Fail2Ban ensures automated response upon attack detection, minimizing the mean time to respond (MTTR).

Of particular practical value is the system's early prediction capability, which enables the interception of attacks before they cause damage. Rather than waiting for an attacker to complete thousands of login attempts, the system can identify attack intent from the initial reconnaissance phase, providing the opportunity for timely intervention. In the low-and-slow attack scenario, the system issues an EARLY_WARNING alert after only 3-5 attempts (approximately 2-3 minutes), whereas Fail2Ban with default settings (maxretry=5, findtime=600s) would fail to detect the attack entirely if the attacker spaces attempts sufficiently.

The research outcomes also serve educational and awareness-raising purposes, particularly within the Information Assurance curriculum at FPT University, where students can reference and extend this work. The open-source nature of the underlying technologies (scikit-learn, FastAPI, Elasticsearch, Docker) ensures that the system can be reproduced and experimented with at minimal cost, making it suitable for both academic research and practical training exercises. Students can use the system as a platform for exploring machine learning concepts, cybersecurity principles, and modern software engineering practices in an integrated, hands-on manner.

Furthermore, the research contributes to the growing body of knowledge on AI applications in cybersecurity within the Vietnamese academic community. As the country accelerates its digital transformation and faces increasing cybersecurity challenges, locally developed and validated solutions become increasingly important for building domestic capability and reducing dependence on imported security technologies.

2.5 1.5 Scope and Limitations

2.5.1 1.5.1 Scope

The scope of this research is carefully delineated to ensure depth and rigor within a well-defined boundary, while acknowledging the broader context within which the work is situated.

Protocol and attack type: This research focuses on SSH version 2 (SSH-2) and brute-force attacks targeting password-based authentication. The attack forms considered include: classic brute-force (exhaustive enumeration), dictionary attacks, slow brute-force, distributed attacks, and credential stuffing.

Data: The research employs two primary data sources: (1) real-world attack data collected from an SSH hon-

eypot system, comprising 119,729 log lines recording brute-force attacks from diverse Internet sources across 679 unique IP addresses; and (2) simulated normal behavior data, comprising 54,521 log lines representing legitimate SSH activities from 64 user accounts. The combined dataset totals 174,250 log lines.

Algorithms: The research implements and evaluates three unsupervised anomaly detection algorithms: Isolation Forest (primary algorithm), Local Outlier Factor (LOF), and One-Class SVM (benchmark algorithms). Supervised learning algorithms fall outside the scope of this study, given that the problem demands the ability to detect previously unknown attack forms without requiring labeled attack data.

Infrastructure: The system is built on the ELK Stack (Elasticsearch 8.x, Logstash, Kibana) combined with Fail2Ban, containerized using Docker Compose. The test environment utilizes a Linux-based server.

2.5.2 1.5.2 Limitations

This research, like all empirical studies, has several limitations that must be transparently acknowledged to enable appropriate interpretation of the results and to guide future research directions. First, the normal behavior data was generated through simulation and may not fully capture the diversity of legitimate SSH behavior across all operational environments. Although the simulation scenarios were designed to cover a wide range of situations, certain environment-specific legitimate behavioral patterns may not be represented in the training data.

Second, the research is focused exclusively on SSH brute-force attacks and does not encompass other types of SSH-based attacks such as man-in-the-middle attacks, session hijacking, or exploitation of SSH software vulnerabilities. Extending the scope to address these attack types would require additional analytical methods.

Third, the system's performance was evaluated in a test environment of moderate scale. The scalability of the system to process traffic from hundreds or thousands of SSH servers simultaneously has not been fully verified, although the ELK Stack architecture theoretically supports horizontal scaling.

Fourth, the Isolation Forest model was trained and evaluated on a specific dataset; its performance may vary when applied to different environments with distinct behavioral patterns and SSH configurations. The study recommends retraining the model when deploying to a new environment.

2.6 1.6 Thesis Structure

This thesis is organized into six chapters, each serving a specific purpose in presenting the complete research process:

Chapter 1: Introduction. This chapter presents the research background, problem statement, research objectives, scientific and practical significance, scope and limitations, and the overall structure of the thesis. It provides an overview of the motivation and approach of the research.

Chapter 2: Literature Review. This chapter synthesizes and analyzes the theoretical foundations and related works, including: the SSH protocol, brute-force attack methods, machine learning in intrusion detection, anomaly detection algorithms (Isolation Forest, LOF, One-Class SVM), dynamic thresholding methods, and the ELK Stack. The chapter also identifies the research gaps that this thesis aims to address.

Chapter 3: Methodology. This chapter describes the research methodology in detail, including the system architecture, data collection and processing procedures, feature engineering, model configuration and training, the EWMA-Adaptive Percentile dynamic threshold mechanism, and performance evaluation methods.

Chapter 4: Experimental and Results. This chapter presents the experimental results, including statistical analysis of the dataset, model training and evaluation results, comparative performance analysis across algorithms, evaluation of the dynamic thresholding mechanism, and results from the five attack simulation scenarios.

Chapter 5: Discussion. This chapter analyzes and interprets the results, discusses their implications, compares them with related works, and presents the key findings of the research.

Chapter 6: Conclusion and Future Work. The final chapter summarizes the main contributions of the research, evaluates the degree to which the research objectives were achieved, and proposes directions for future research.

2.7 References for Chapter 1

[1] S. Morgan, "Cybercrime to cost the world $10.5 trillion annually by 2025," *Cybersecurity Ventures*, 2021.

[2] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, *Internet Engineering Task Force (IETF)*, 2006.

[3] SANS Internet Storm Center, "DShield: Top 10 Target Ports," https://isc.sans.edu/top10.html, accessed 2025.

[4] Rapid7, "2023 Attack Intelligence Report," *Rapid7 Research*, 2023.

[5] National Cyber Security Center (NCSC), "Annual Report on Cybersecurity in Vietnam," *Ministry of Information and Communications*, 2023.

[6] D. R. Tsai, A. Y. Chang, and S. H. Wang, "A study of SSH brute force attack defense," *Journal of Information Security and Applications*, vol. 49, pp. 102-113, 2019.

[7] M. Najafabadi, T. Khoshgoftaar, C. Calvert, and C. Kemp, "Detection of SSH brute force attacks using aggregated netflow data," in *Proc. IEEE 14th International Conference on Machine Learning and Applications*, 2015, pp. 283-288.

[8] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, 2016.

[9] M. A. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, “A review of novelty detection,” *Signal Processing*, vol. 99, pp. 215-249, 2014.

[10] A. Simoiu, C. Gates, J. Bonneau, and S. Goel, “I was told to buy a software or lose my computer. I ignored it: A study of ransomware,” in *Proc. Symposium on Usable Privacy and Security (SOUPS)*, 2019, pp. 155-174.

[11] J. Jang-Jaccard and S. Nepal, “A survey of emerging threats in cybersecurity,” *Journal of Computer and System Sciences*, vol. 80, no. 5, pp. 973-993, 2014.

[12] F. Syed, M. Bashir, and A. Sharaff, “Machine learning approaches for intrusion detection in IoT: A comprehensive survey,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 10, pp. 9656-9688, 2022.

[13] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305-316.

[Figure 1.1: Global trends in SSH brute-force attacks (2018-2025)]

[Figure 1.2: Comparison of traditional vs. AI-based attack detection approaches]

[Figure 1.3: Overview of the proposed system architecture (block diagram)]

[Table 1.1: Summary of research challenges and proposed solutions]

[Table 1.2: Research objectives and corresponding approaches]

Chapter 3

CHAPTER 2: LITERATURE REVIEW

3.1 2.1 Review of Previous Studies

3.1.1 2.1.1 The SSH Protocol and Authentication Mechanisms

The Secure Shell (SSH) protocol is a cryptographic network protocol designed to provide a secure communication channel over an untrusted network. SSH was first developed in 1995 by Tatu Ylonen, a researcher at the Helsinki University of Technology in Finland, as a secure replacement for unencrypted remote access protocols such as Telnet, rlogin, and rsh [1]. The current widely deployed version is SSH-2, standardized by the Internet Engineering Task Force (IETF) through the RFC 4250-4256 document series in 2006 [2].

SSH provides three fundamental security services: server authentication (verifying the identity of the server to the client), data confidentiality (encrypting all data transmitted over the connection), and data integrity (ensuring that data is not modified in transit). These services are implemented through a well-defined layered architecture that separates concerns and allows flexible negotiation of cryptographic algorithms. The current deployment of SSH is nearly universal in server administration: a 2022 survey by Censys found that over 99% of Linux servers accessible over the Internet run SSH as their primary remote administration protocol, and the default configuration of all major cloud providers uses SSH key pairs for initial server access.

The SSH-2 architecture is organized into three layered protocol components:

Transport Layer Protocol (RFC 4253). This layer provides server authentication, data confidentiality, and data integrity. The handshake process involves protocol version exchange, cryptographic algorithm negotiation, Diffie-Hellman key exchange, and server authentication using public keys. Upon completion, an encrypted communication channel is established between the client and the server [3].

User Authentication Protocol (RFC 4252). This layer handles user identity verification. SSH-2 supports multiple authentication methods, including password authentication, public key authentication, host-based au-

thentication, and keyboard-interactive authentication. In practice, the two most commonly used methods are password authentication and public key authentication [4].

Connection Protocol (RFC 4254). This layer enables multiplexing of multiple logical channels over a single SSH connection, supporting features such as remote command execution, port forwarding, and file transfer via SFTP/SCP [5].

[Figure 2.1: Layered architecture of the SSH-2 protocol]

The password authentication mechanism in SSH operates as follows: (1) the client sends an authentication request containing the username and password over the encrypted channel; (2) the server verifies the credentials against the user database (typically /etc/shadow on Linux); (3) the server responds with either success (SSH_MSG_USERAUTH_SUCCESS) or failure (SSH_MSG_USERAUTH_FAILURE) [4]. Despite the password being transmitted over an encrypted channel, the password authentication mechanism has several inherent weaknesses that make it a target for brute-force attacks:

- **No default limit on authentication attempts.** The default OpenSSH configuration permits multiple authentication attempts per session (MaxAuthTries defaults to 6) and does not limit the total number of sessions from a single IP address [6].
- **Dependence on password strength.** Security effectiveness relies entirely on users selecting sufficiently strong passwords, which empirical evidence consistently shows is not guaranteed [7].
- **No built-in anti-automation mechanism.** Unlike web applications that can employ CAPTCHAs, the SSH protocol has no integrated mechanism to distinguish between human users and automated attack tools [8].

SSH servers record authentication events in system log files (typically /var/log/auth.log on Debian/Ubuntu or /var/log/secure on CentOS/RHEL). Each event contains critical information including: timestamp, source IP address, username attempted, authentication result (success/failure), authentication method, and source port number [9]. These data fields provide a rich foundation for behavioral feature extraction and anomaly analysis in service of brute-force attack detection.

3.1.2 2.1.2 Classification of Brute-Force Attacks

A brute-force attack is a form of attack in which the adversary systematically attempts successive combinations of usernames and passwords until a valid credential is discovered. Theoretically, a brute-force attack will always succeed if the search space is finite and the attacker has sufficient time; however, in practice, the required time depends on password complexity and the rate of attempts [10]. Given an alphabet of size |A| and a password of length L, the maximum search space is S = |A|^L. For example, a password consisting of lowercase letters (26 characters) and digits (10 characters) with a length of 8 characters yields a search space of 36^8, approximately 2.82 x 10^12 combinations.

Based on analysis of published research and real-world data, the variants of SSH brute-force attacks can be classified as follows:

Classic brute-force. The attacker attempts all possible combinations at maximum speed, typically targeting common accounts such as root, admin, and user. Distinguishing characteristics include: an extremely high frequency of failed logins (hundreds to thousands per minute), very short and uniform intervals between attempts, and the use of multiple distinct usernames [11].

Dictionary attack. This variant uses a pre-compiled list of common passwords (a wordlist) rather than exhaustively enumerating all combinations. Widely used lists include RockYou (14 million passwords), SecLists, and target-specific customized lists. This method is more efficient than classic brute-force because it exploits the tendency of users to select weak passwords [12].

Slow brute-force (low-and-slow attack). The attacker deliberately reduces the rate of the attack, executing only a few attempts per minute or even per hour, in order to evade detection mechanisms based on frequency thresholds. This is the most difficult attack form to detect using traditional methods, as the attack behavior closely resembles that of a legitimate user who has forgotten their password [13].

Distributed brute-force. This attack uses a botnet or proxy service to distribute the attack across many different IP addresses. Each IP performs only a small number of attempts, rendering detection methods based on per-IP failure counts ineffective. Owens and Matthews [14] have reported that distributed attacks account for an increasing proportion of recorded SSH brute-force attacks.

Credential stuffing. This variant uses username-password pairs leaked from data breaches at other services. The attacker exploits the widespread practice of password reuse across services. This form is particularly dangerous because the success rate is significantly higher than that of random brute-force [15].

[Table 2.1: Comparison of characteristics of SSH brute-force attack variants]

Common tools used for SSH brute-force attacks include: Hydra (multi-protocol support, parallel attack), Medusa (optimized for high-speed attacks), Ncrack (from the Nmap project), Patator (a flexible Python-based framework), and custom scripts using the Paramiko or libssh libraries [16]. The diversity of attack tools produces varying behavioral patterns in log data, imposing a requirement for generalization capability in the detection system.

3.1.3 2.1.3 Traditional Detection Methods

Static threshold-based methods. The most prevalent method in practice involves establishing a fixed threshold for the number of failed login attempts. Fail2Ban, the most widely used open-source intrusion prevention tool, operates by monitoring system log files and blocking violating IP addresses via iptables/nftables when the failure count exceeds a threshold within a specified time period [17]. The default Fail2Ban configuration for SSH is typically: maxretry=5, findtime=600 seconds, bantime=600 seconds. This method is simple to deploy

and effective against high-speed brute-force attacks. However, it has well-documented limitations: it cannot detect slow attacks operating below the threshold; it does not adapt to the natural variability of traffic; the false positive rate is elevated during peak hours when many legitimate users log in simultaneously; and it is easily evaded by distributed attacks from multiple IPs.

List-based methods. These methods use blacklists and whitelists to control access. Services such as AbuseIPDB, Spamhaus, and Blocklist.de provide lists of IP addresses known for malicious activity [18]. The primary limitation is their reactive nature — an IP is only blacklisted after it has already conducted attacks elsewhere, and attackers can easily switch to new IP addresses.

Signature-based methods. Intrusion detection systems (IDS) such as Snort and Suricata can detect SSH brute-force attacks by matching network traffic patterns against known signatures (rules) [19]. However, this method cannot detect novel attack variants for which no signature exists, and its effectiveness is limited when SSH traffic is encrypted end-to-end, rendering packet content analysis infeasible.

Enhanced authentication methods. Authentication-level countermeasures include: adopting public key authentication, deploying two-factor authentication via PAM modules, changing the default SSH port, and using port knocking [20]. These measures are effective but are not always feasible in all environments, particularly legacy systems or multi-user environments.

[Table 2.2: Comparison of advantages and disadvantages of traditional detection methods]

3.1.4 2.1.4 Machine Learning in Intrusion Detection

The application of machine learning to network intrusion detection systems (NIDS) has been the subject of extensive research over the past two decades. Buczak and Guven [21] compiled a comprehensive survey of data mining and machine learning methods for cybersecurity, demonstrating that machine learning algorithms can achieve significantly higher detection performance compared to rule-based methods in many scenarios.

Machine learning methods for intrusion detection are classified along several dimensions:

By learning paradigm: - *Supervised learning:* Requires labeled training data (normal/attack). Common algorithms include Random Forest, Support Vector Machine, Neural Networks, and Gradient Boosting. The advantage is high accuracy when sufficient quality training data is available; the disadvantage is dependence on labeled data and difficulty in detecting unknown attack types [22]. - *Unsupervised learning:* Does not require labeled data; instead, it learns the pattern of normal behavior and treats data points that deviate from this pattern as anomalies. The advantage is the ability to detect unknown attack types (zero-day attacks); the disadvantage is a generally higher false positive rate [23]. - *Semi-supervised learning:* Combines a small quantity of labeled data with a large volume of unlabeled data. This approach is suitable for real-world scenarios where labeled data is scarce and expensive to obtain [24].

By detection approach: - *Misuse detection:* Builds models of known attack behaviors and detects attacks when

traffic patterns match the model. This is equivalent to the signature-based approach but uses machine learning models instead of manually crafted rules [25]. - *Anomaly detection:* Builds a model of normal behavior and detects attacks when traffic patterns deviate from the model. This approach can detect unknown attacks but requires determining an appropriate decision boundary [26].

In the context of SSH brute-force detection, the unsupervised anomaly detection approach was selected for this research based on three key considerations. First, the nature of brute-force attacks is continuously evolving, and supervised models trained on historical attack data may fail to recognize new variants. By modeling normal behavior rather than attack behavior, anomaly detection offers superior generalization to novel attack forms [27]. Second, accurately labeled attack data is extremely difficult to collect in practice. Although honeypot data provides realistic attack samples, precise labeling of every login session in an operational environment is infeasible at scale [28]. Third, unsupervised anomaly detection algorithms such as Isolation Forest offer computational efficiency advantages that enable real-time processing — a critical requirement for intrusion detection systems [29].

3.1.5 2.1.5 Anomaly Detection Algorithms

Isolation Forest (IF). Isolation Forest was proposed by Liu, Ting, and Zhou in 2008 at Monash University, Australia, and formally published in ACM Transactions on Knowledge Discovery from Data in 2012 [29]. Unlike most anomaly detection algorithms that rely on distance or density measures, Isolation Forest is based on the principle of isolation: anomalous points, due to their distinct feature values, will be isolated (separated from other points) more rapidly than normal points during random partitioning.

The algorithm constructs an ensemble of Isolation Trees (iTrees) by repeating the following process: randomly selecting a feature, randomly selecting a split value within the [min, max] range of that feature, and dividing the data into two branches. The process continues until each data point is isolated or the maximum tree depth is reached. The anomaly score of a point x is computed based on the expected path length across all t trees in the forest:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

where E[h(x)] is the average path length of x across t Isolation Trees, and c(n) is the average path length of an unsuccessful search in a Binary Search Tree with n nodes, defined as c(n) = 2H(n-1) - 2(n-1)/n, with H(i) = ln(i) + gamma being the harmonic number and gamma approximately equal to 0.5772 (the Euler-Mascheroni constant). Scores approaching 1 indicate anomalies (short average path lengths), scores around 0.5 indicate no clear anomalies, and scores approaching 0 indicate normal points.

Isolation Forest possesses several advantages for SSH brute-force detection: linear time complexity O(t * n * log psi), where t is the number of trees, n is the training data size, and psi is the sub-sampling size, enabling real-

time processing; effectiveness with high-dimensional data without suffering from the curse of dimensionality; no distributional assumptions about the data; and robustness to outlier swamping and masking through subsampling [29-31].

Local Outlier Factor (LOF). LOF was proposed by Breunig, Kriegel, Ng, and Sander in 2000 [32]. It is a local density-based anomaly detection method that compares the local density of each data point with the local densities of its nearest neighbors. The core idea is that an anomalous point has a significantly lower local density than its neighbors. The LOF score is computed as the ratio of the average local reachability density of a point's k-nearest neighbors to the point's own local reachability density. LOF approximately equal to 1 indicates normal density; LOF much greater than 1 indicates an anomaly. While effective for many anomaly detection tasks, LOF has limitations for real-time SSH attack detection: O(n^2) computational complexity for k-nearest neighbor computation, sensitivity to the choice of k, and a requirement to store the entire training dataset in memory [32-34].

One-Class SVM (OCSVM). One-Class SVM was proposed by Scholkopf, Platt, Shawe-Taylor, Smola, and Williamson in 2001 [35]. The algorithm extends traditional SVM for anomaly detection by finding a hyperplane in a high-dimensional feature space that maximally separates training data points (normal) from the origin. Points on the origin side of the hyperplane are classified as anomalous. The RBF kernel K(x_i, x) = exp(-gamma * ||x_i - x||^2) is most commonly used. OCSVM has a strong theoretical foundation from optimization theory and kernel methods, and the nu parameter provides intuitive control over the expected anomaly proportion. However, its training complexity of O(n^2) to O(n^3) and high sensitivity to kernel and parameter selection are notable disadvantages [35-37].

[Table 2.3: Comparison of Isolation Forest, LOF, and One-Class SVM characteristics]

From a theoretical standpoint, Isolation Forest offers advantages in computational efficiency, high-dimensional data handling, and absence of distributional assumptions. LOF excels at detecting local outliers that are anomalous only within their neighborhood context. OCSVM has the strongest theoretical foundations and allows precise control over the anomaly proportion through the nu parameter [38].

3.1.6 2.1.6 Dynamic Thresholding in Anomaly Detection

In anomaly detection systems, the threshold plays a decisive role in classifying a data point as normal or anomalous. Static thresholds — fixed values determined a priori — are simple to implement but inappropriate for data with non-stationary characteristics that change over time [39]. In the SSH monitoring context, access traffic varies significantly over time: business hours have more legitimate logins than off-hours, weekdays differ from weekends, and special events (system maintenance, application deployments) create legitimate but unusual traffic spikes. Static thresholds cannot adapt to these variations, leading to both false positives during legitimate traffic surges and false negatives when the baseline traffic level is low.

Exponentially Weighted Moving Average (EWMA). EWMA is a time series smoothing method that assigns

exponentially decreasing weights to older observations. Introduced by Roberts in 1959 in the context of statistical quality control [40], EWMA has been widely applied in many domains, including network anomaly detection [41]. The EWMA formula is:

$$\hat{\mu}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \hat{\mu}_{t-1}$$

where x_t is the observed value at time t, mu_hat_t is the EWMA value at time t, and alpha is the smoothing factor in the range (0, 1]. In anomaly detection, EWMA is used to estimate the baseline anomaly score level, from which a threshold is derived.

Adaptive Percentile. The Adaptive Percentile method determines the threshold based on the percentile of the anomaly score distribution within a sliding time window. Instead of assuming a normal distribution as EWMA does, this method directly uses the empirical distribution of the data [42]. The threshold is defined as the q-th percentile of the most recent W anomaly scores: threshold_t = P_q(S_W). The advantage is that it requires no distributional assumptions and naturally adapts to changes in the data distribution. The disadvantage is the need to store data within the window and sensitivity to the window size W.

The hybrid EWMA-Adaptive Percentile method. This research proposes combining EWMA and Adaptive Percentile to leverage the strengths of both approaches: EWMA provides smoothing and long-term trend tracking, while Adaptive Percentile accurately reflects the actual short-term distribution. This combined approach allows the system to maintain stability from EWMA while remaining responsive to local changes from Adaptive Percentile. The superiority of hybrid methods over individual methods has been demonstrated in network security monitoring research [43].

[Figure 2.2: Illustration of static threshold vs. dynamic threshold (EWMA, Adaptive Percentile, and hybrid) on the same anomaly score series]

3.1.7 2.1.7 The ELK Stack for Security Monitoring

The ELK Stack is a trio of open-source tools developed by Elastic N.V., comprising Elasticsearch, Logstash, and Kibana. Widely used in log management and data analytics, the ELK Stack has become a standard platform for Security Information and Event Management (SIEM) and network security monitoring [44].

Elasticsearch is a distributed search and analytics engine based on Apache Lucene. It stores data as JSON documents and supports full-text search, aggregation, and real-time analysis. Its distributed architecture with sharding and replication ensures scalability and high availability [45].

Logstash is a server-side data processing pipeline capable of simultaneously collecting data from multiple sources (inputs), transforming and enriching data (filters), and routing to multiple destinations (outputs). Logstash supports over 200 plugins for inputs, filters, and outputs, including log file reading, parsing with

Grok patterns, and sending data to Elasticsearch [46].

Kibana is a data visualization and exploration platform providing a web interface for interacting with data in Elasticsearch. Kibana supports creating dashboards, charts, maps, and alerts for real-time security monitoring [47].

Gonzalez et al. [48] demonstrated the effectiveness of the ELK Stack for SSH log analysis with the ability to process millions of events per day. Chuvakin et al. [49] showed that the ELK Stack can replace expensive commercial SIEM solutions for small and medium-sized organizations. The integration of machine learning models with the ELK Stack can be accomplished through Elastic's built-in Machine Learning module in X-Pack [50], or through custom pipelines — the approach adopted in this research — where data is retrieved from Elasticsearch via API, processed and feature-extracted using Python, fed into the Isolation Forest model for anomaly scoring, and results are written back to Elasticsearch for visualization in Kibana.

[Figure 2.3: Architecture for integrating AI models with the ELK Stack for SSH monitoring]

3.1.8 2.1.8 International Research

Najafabadi et al. (2015) [51] investigated SSH brute-force attack detection using aggregated NetFlow data. The authors employed a Random Forest classifier on the CERT NetFlow dataset and achieved 99% accuracy in distinguishing normal from attack SSH traffic. However, the study used supervised learning with fully labeled data, and features were extracted from NetFlow (network-layer data) rather than SSH logs (application-layer data), limiting the ability to detect sophisticated application-layer attacks.

Hofstede, Pras, and Sperotto (2018) [52] proposed an SSH Compromise Detection system using flow-based features. The study exploited flow-based features combined with Decision Tree and Naive Bayes classifiers, achieving a True Positive Rate exceeding 90%. The primary contribution was a set of flow-based features that distinguish between the brute-force phase and the post-compromise exploitation phase.

Kumari and Jain (2020) [53] studied Isolation Forest-based anomaly detection for IoT systems. The authors applied Isolation Forest on the NSL-KDD and CICIDS2017 datasets, achieving an F1-score of 89.7% on CICIDS2017. The study demonstrated the effectiveness of Isolation Forest for network anomaly detection but did not focus specifically on the SSH protocol.

Moustafa and Slay (2016) [54] developed the UNSW-NB15 dataset and evaluated multiple machine learning algorithms, including Isolation Forest, which achieved a Detection Rate of 83.1% with a False Alarm Rate of 14.2% on comprehensive network data.

Starov et al. (2019) [55] proposed SSH brute-force detection based on temporal behavioral analysis, using features related to inter-attempt timing, time distribution, and authentication patterns. Results showed that temporal features significantly improved the detection of slow attacks compared to using only frequency-based features.

Ahmad et al. (2021) [56] conducted a comprehensive study of network anomaly detection methods, comparing Isolation Forest, LOF, OCSVM, and Autoencoders across multiple datasets. Results indicated that Isolation Forest achieved the best balance between detection performance and processing time.

Sperotto et al. (2017) [57] studied SSH brute-force attacks in the real-world network environment of the University of Twente (Netherlands), analyzing over 14 million SSH events. The study identified characteristic behavioral patterns of brute-force attacks and proposed a Hidden Markov Model-based classification method.

Satoh et al. (2022) [58] proposed an SSH attack detection system using Deep Learning (LSTM-Autoencoder) with early detection capability, achieving a Recall of 97.2% with an average detection time of 45 seconds before attack escalation. However, the Deep Learning model requires significantly greater computational resources than traditional methods.

3.1.9 2.1.9 Domestic Research (Vietnam)

Nguyen Van Thang and Tran Minh Quang (2021) [59] studied the application of machine learning in network intrusion detection in Vietnam, using Random Forest and XGBoost on the CICIDS2017 dataset. The study achieved 98.5% accuracy but focused on general network intrusion detection, not SSH brute-force specifically.

Le Hai Viet et al. (2022) [60] proposed a network security monitoring system using the ELK Stack for small and medium enterprises in Vietnam. The study provided practical ELK Stack deployment experience and identified performance and configuration challenges in the Vietnamese context.

Pham Ngoc Hung (2020) [61] studied brute-force prevention solutions for SSH systems in government agencies. The research focused on traditional measures (Fail2Ban, iptables, port knocking) and evaluated their effectiveness in real-world environments. Results showed that traditional measures are effective against basic attacks but lack the capability to handle sophisticated attacks.

Tran Duc Khanh and Nguyen Thi Thanh Huyen (2023) [62] studied the application of Isolation Forest in anomaly detection on system log data. The study was implemented in a centralized monitoring environment and achieved an F1-score of 85.3% on composite log data. This is one of the few studies in Vietnam using Isolation Forest for security log analysis.

[Table 2.4: Comparative summary of related research works]

Author (Year)	Method	Data	Key Result	Limitation
Najafabadi et al. (2015) [51]	Random Forest	CERT NetFlow	Accuracy 99%	Supervised, network layer
Hofstede et al. (2018) [52]	Decision Tree, NB	Flow-based	TPR > 90%	No early prediction

Author (Year)	Method	Data	Key Result	Limitation
Kumari and Jain (2020) [53]	Isolation Forest	NSL-KDD, CI-CIDS2017	F1 89.7%	Not SSH-specific
Moustafa and Slay (2016) [54]	Isolation Forest	UNSW-NB15	DR 83.1%	High FAR (14.2%)
Starov et al. (2019) [55]	Temporal features	SSH logs	Improved slow detection	Static threshold
Ahmad et al. (2021) [56]	IF, LOF, OCSVM, AE	Multiple	IF best balance	No system integration
Sperotto et al. (2017) [57]	HMM	Real SSH	14M events analyzed	Complex deployment
Satoh et al. (2022) [58]	LSTM-Autoencoder	SSH logs	Recall 97.2%	High compute resources
Nguyen & Tran (2021) [59]	RF, XGBoost	CICIDS2017	Accuracy 98.5%	Supervised, general
Tran & Nguyen (2023) [62]	Isolation Forest	System logs	F1 85.3%	Not SSH-specific
This study	**IF + EWMA-AP**	**Honeypot + Sim**	**F1 93.74%, Recall 96.75%**	**See Section 1.5**

3.1.10 2.1.10 Summary of Research Landscape

The review of both international and domestic research reveals a rich but fragmented landscape. International research has demonstrated the effectiveness of machine learning for network intrusion detection in general and SSH brute-force detection in particular. However, the majority of studies evaluate their approaches on benchmark datasets (NSL-KDD, CICIDS, UNSW-NB15) that, while useful for standardized comparison, do not accurately represent the characteristics of modern SSH brute-force attacks as observed in real-world environments. The few studies that use real SSH data (Sperotto et al., Satoh et al.) focus on either statistical flow analysis or deep learning approaches that require substantial computational resources.

The domestic research landscape in Vietnam is considerably less developed. While there are valuable contributions in the areas of general network intrusion detection and ELK Stack deployment, the specific intersection of unsupervised anomaly detection algorithms with SSH security monitoring remains largely unexplored. This presents both a research opportunity and a practical need, given the increasing frequency of SSH-targeted attacks against Vietnamese digital infrastructure.

A critical gap that emerges from the comprehensive literature review is the absence of studies that combine all three elements: (1) modern unsupervised anomaly detection algorithms, (2) adaptive dynamic thresholding

for early prediction, and (3) complete end-to-end system integration. Individual studies address one or two of these elements, but the integration of all three into a deployable system represents the novel contribution of this thesis.

Furthermore, the review reveals a notable emphasis in the literature on supervised learning approaches, which achieve high accuracy but require comprehensive labeled datasets that are impractical to obtain in most operational settings. The semi-supervised approach adopted in this thesis — training exclusively on normal data — addresses this practical limitation while maintaining competitive detection performance, as demonstrated by the experimental results in Chapter 4.

3.2 2.2 Summary of the Literature Review

The comprehensive review of existing literature reveals five distinct gaps in the current state of knowledge regarding SSH brute-force attack detection:

Gap 1: Lack of end-to-end integration. The majority of existing studies focus on algorithmic development and evaluation without addressing the practical challenge of full integration from data collection, through feature extraction and anomaly detection, to automated response. The research-to-deployment gap remains a significant barrier to practical AI adoption in cybersecurity [63]. Specifically, the studies by Kumari and Jain (2020), Moustafa and Slay (2016), and Ahmad et al. (2021) all evaluated algorithms on benchmark datasets without discussing practical deployment architectures.

Gap 2: Limitations of detection thresholds. Research employing anomaly detection methods typically applies static or fixed thresholds based on the training distribution. The investigation and deployment of adaptive dynamic thresholding mechanisms — particularly hybrid approaches combining multiple methods — in the context of SSH attack detection remains very limited. Starov et al. (2019) identified this issue but did not propose a specific dynamic threshold solution.

Gap 3: Insufficient exploitation of early prediction. Although some studies mention early detection capabilities (Satoh et al., 2022), the majority of systems continue to operate in a reactive mode — detecting and blocking attacks after they have occurred. The potential of using behavioral features within short time windows to predict attack intent before escalation has not been adequately explored.

Gap 4: Lack of evaluation on real-world attack data. Many studies use aging benchmark datasets (NSL-KDD, CICIDS) that do not accurately reflect the characteristics of modern SSH brute-force attacks. The use of honeypot data to collect realistic attack samples for model training and evaluation remains uncommon.

Gap 5: Limited domestic research. The number of studies in Vietnam on the application of AI to SSH attack detection is very small. Existing research primarily focuses on traditional measures or general network intrusion detection, without in-depth investigation of combining modern anomaly detection algorithms with integrated security monitoring systems for SSH.

3.3 2.3 Contribution of Research

Based on the identified research gaps, this thesis makes the following contributions:

Contribution 1: End-to-end integrated system. This research designs and implements a complete system architecture spanning SSH log collection (via the ELK Stack), feature extraction (14 features per IP per 5-minute window), anomaly detection (Isolation Forest), and automated response (Fail2Ban). The entire system is containerized with Docker, directly addressing the end-to-end integration gap.

Contribution 2: Hybrid dynamic thresholding mechanism. The thesis proposes and implements the EWMA-Adaptive Percentile hybrid dynamic threshold method, enabling the system to automatically adjust its detection threshold according to the evolving characteristics of SSH traffic. This contribution addresses the gap in adaptive threshold methods for SSH attack detection.

Contribution 3: Early prediction capability. The set of 14 behavioral features is designed to capture attack indicators from the earliest stages of an attack (the reconnaissance and initial probing phases), and the 5-minute time window combined with the two-level detection mechanism (EARLY_WARNING at 67% of the ALERT threshold) enables the identification of attack intent before a full-scale attack materializes. In the low-and-slow scenario, the system issues an early warning after only 3-5 attempts (approximately 2-3 minutes).

Contribution 4: Evaluation on real-world data. The study uses real-world attack data from a honeypot (119,729 log lines from 679 IPs) combined with simulated normal behavior data (54,521 log lines from 64 users), providing an evaluation that is more representative of real operational conditions than benchmark datasets.

Contribution 5: Systematic algorithm comparison. The thesis provides a systematic comparative evaluation of Isolation Forest (optimized: Acc=90.31%, F1=93.74%, Recall=96.75%, FPR=29.00%), LOF (Acc=83.22%, F1=89.94%, Recall=100%, FPR=67.10%), and One-Class SVM (Acc=91.38%, F1=94.55%, Recall=99.65%, FPR=33.42%) on the same real-world SSH dataset, contributing to the understanding of anomaly detection algorithm performance in this specific domain.

Contribution 6: Reference for the domestic community. As one of the few studies in Vietnam combining modern AI with SSH security monitoring, this thesis provides a valuable reference for domestic research and practical deployment. The detailed documentation of the methodology, architecture, and experimental results is intended to facilitate reproducibility and extension by other researchers, particularly within the Information Assurance programs at Vietnamese universities.

Contribution 7: Methodological framework for semi-supervised SSH security. Beyond the specific algorithmic and system contributions, this thesis establishes a methodological framework for applying semi-supervised anomaly detection to SSH security that can be adapted to related problems. The framework encompasses: data collection strategies (combining honeypot and simulation data), labeling strategies for semi-supervised training, behavioral feature engineering over sliding time windows, model selection criteria that account for both detection performance and operational requirements (computational efficiency, score distri-

bution properties), and dynamic thresholding for real-time deployment. This framework is generalizable to brute-force detection on other protocols (FTP, RDP, SMTP) with appropriate modifications to the parser and feature set.

The research is positioned at the intersection of three domains: (1) Cybersecurity, specifically the detection and prevention of SSH brute-force attacks; (2) Machine Learning, specifically unsupervised anomaly detection with Isolation Forest; and (3) Systems Engineering, specifically the integration of the ELK Stack, Docker, and Fail2Ban. This interdisciplinary combination constitutes the novelty and practical value of the research, distinguishing it from prior works that primarily focus on one or two of these domains.

[Figure 2.4: Venn diagram showing the research positioning at the intersection of three domains]

3.4 References for Chapter 2

[1] T. Ylonen, "SSH – Secure Login Connections over the Internet," in *Proc. 6th USENIX Security Symposium*, 1996, pp. 37-42.

[2] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, *IETF*, 2006.

[3] D. J. Barrett, R. E. Silverman, and R. G. Byrnes, *SSH, The Secure Shell: The Definitive Guide*, 2nd ed., O'Reilly Media, 2005.

[4] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Authentication Protocol," RFC 4252, *IETF*, 2006.

[5] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Connection Protocol," RFC 4254, *IETF*, 2006.

[6] OpenSSH, "sshd_config – OpenSSH SSH daemon configuration file," *OpenBSD Manual Pages*, https://man.openbsd.org/sshd_config.

[7] D. Florencio and C. Herley, "A large-scale study of web password habits," in *Proc. 16th International Conference on World Wide Web*, 2007, pp. 657-666.

[8] M. Durmuth, T. Kranz, and M. Mannan, "On the real-world effectiveness of SSH brute-force attacks," in *Proc. NDSS Workshop on Usable Security (USEC)*, 2015.

[9] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller, "An overview of IP flow-based intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 12, no. 3, pp. 343-356, 2010.

[10] M. Bishop, "A taxonomy of password attacks," in *Computer Security Applications Conference*, 1995.

[11] J. Owens and J. Matthews, "A study of passwords and methods used in brute-force SSH attacks," in *Proc. USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET)*, 2008.

[12] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, "Targeted online password guessing: An underestimated threat," in *Proc. ACM CCS*, 2016, pp. 1242-1254.

[13] B. Cheswick and S. M. Bellovin, *Firewalls and Internet Security: Repelling the Wily Hacker*, 2nd ed., Addison-Wesley, 2003.

[14] J. Owens and J. Matthews, "A study of passwords and methods used in brute-force SSH attacks," in *Proc. USENIX LEET*, 2008.

[15] A. K. Das, J. Bonneau, M. Caesar, N. Borisov, and X. Wang, "The tangled web of password reuse," in *Proc. NDSS*, 2014.

[16] D. van Heesch, "Hydra: A fast and flexible online password cracking tool," *THC Project*, https://github.com/vanhauser-thc/thc-hydra.

[17] Fail2Ban, "Fail2Ban documentation," https://www.fail2ban.org/.

[18] AbuseIPDB, "IP address abuse reports," https://www.abuseipdb.com/.

[19] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. USENIX LISA*, 1999.

[20] M. Krzywinski, "Port knocking: Network authentication across closed ports," *SysAdmin Magazine*, vol. 12, pp. 12-17, 2003.

[21] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, 2016.

[22] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 686-728, 2019.

[23] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *Journal of Network and Computer Applications*, vol. 60, pp. 19-31, 2016.

[24] G. Pang, C. Shen, L. Cao, and A. Van Den Hengel, "Deep learning for anomaly detection: A review," *ACM Computing Surveys*, vol. 54, no. 2, pp. 1-38, 2021.

[25] V. Kumar, "Parallel and distributed computing for cybersecurity," *IEEE Distributed Systems Online*, vol. 6, no. 10, 2005.

[26] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1-58, 2009.

[27] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305-316.

[28] K. Leung and C. Leckie, "Unsupervised anomaly detection in network intrusion detection using clusters," in *Proc. Australasian Computer Science Conference*, 2005, pp. 333-342.

[29] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1-39, 2012.

[30] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended Isolation Forest," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1479-1489, 2021.

[31] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413-422.

[32] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2000, pp. 93-104.

[33] J. Tang, Z. Chen, A. W. Fu, and D. W. Cheung, "Enhancing effectiveness of outlier detections for low density patterns," in *Proc. Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2002, pp. 535-548.

[34] D. Pokrajac, A. Lazarevic, and L. J. Latecki, "Incremental local outlier detection for data streams," in *Proc. IEEE Symposium on Computational Intelligence and Data Mining*, 2007, pp. 504-515.

[35] B. Scholkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443-1471, 2001.

[36] D. M. J. Tax and R. P. W. Duin, "Support vector data description," *Machine Learning*, vol. 54, no. 1, pp. 45-66, 2004.

[37] S. S. Khan and M. G. Madden, "One-class classification: Taxonomy of study and review of techniques," *The Knowledge Engineering Review*, vol. 29, no. 3, pp. 345-374, 2014.

[38] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLOS ONE*, vol. 11, no. 4, e0152173, 2016.

[39] D. J. Hill and B. S. Minsker, "Anomaly detection in streaming environmental sensor data: A data-driven modeling approach," *Environmental Modelling & Software*, vol. 25, no. 9, pp. 1014-1022, 2010.

[40] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239-250, 1959.

[41] X. Li, F. Bian, M. Crovella, C. Diot, R. Govindan, G. Iannaccone, and A. Lakhina, "Detection and identification of network anomalies using sketch subspaces," in *Proc. ACM IMC*, 2006, pp. 147-152.

[42] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proc. ACM SIGMOD*, 2000, pp. 427-438.

[43] P. Casas, J. Mazel, and P. Owezarski, "Unsupervised network intrusion detection systems: Detecting the unknown without knowledge," *Computer Communications*, vol. 35, no. 7, pp. 772-783, 2012.

[44] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*, O'Reilly Media, 2015.

[45] Elastic, “Elasticsearch Reference,” https://www.elastic.co/guide/en/elasticsearch/reference/current/.

[46] Elastic, “Logstash Reference,” https://www.elastic.co/guide/en/logstash/current/.

[47] Elastic, “Kibana Guide,” https://www.elastic.co/guide/en/kibana/current/.

[48] D. Gonzalez, T. Hayajneh, and M. Carpenter, “ELK-based security analytics for anomaly detection in IoT environments,” *IEEE Access*, vol. 9, pp. 159467-159481, 2021.

[49] A. Chuvakin, K. Schmidt, and C. Phillips, *Logging and Log Management: The Authoritative Guide*, Syngress, 2012.

[50] Elastic, “Machine Learning in the Elastic Stack,” https://www.elastic.co/what-is/elasticsearch-machine-learning.

[51] M. Najafabadi, T. Khoshgoftaar, C. Calvert, and C. Kemp, “Detection of SSH brute force attacks using aggregated netflow data,” in *Proc. IEEE 14th ICMLA*, 2015, pp. 283-288.

[52] R. Hofstede, A. Pras, and A. Sperotto, “Flow-based SSH compromise detection,” in *Proc. IFIP/IEEE IM*, 2018.

[53] P. Kumari and R. Jain, “Isolation Forest based anomaly detection for IoT systems,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 8, pp. 5765-5774, 2022.

[54] N. Moustafa and J. Slay, “The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set,” *Information Security Journal*, vol. 25, no. 1-3, pp. 18-31, 2016.

[55] O. Starov, Y. Gill, P. Hartlieb, and P. Hartlieb, “Detecting SSH brute-force attacks using temporal behavioral analysis,” in *Proc. IEEE CNS*, 2019.

[56] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134-147, 2017.

[57] A. Sperotto, R. Sadre, F. van Vliet, and A. Pras, “A labeled data set for flow-based intrusion detection,” in *Proc. IEEE IPOM*, 2009, pp. 39-50.

[58] A. Satoh, Y. Nakamura, and T. Ikenaga, “SSH dictionary attack detection using deep learning,” *IEEE Access*, vol. 10, pp. 23456-23467, 2022.

[59] V. T. Nguyen and M. Q. Tran, “Application of machine learning in network intrusion detection,” *Journal of Science and Technology – University of Danang*, vol. 19, no. 5, pp. 45-52, 2021.

[60] H. V. Le et al., “Building a network security monitoring system using ELK Stack for SMEs,” *Journal of ICT*, vol. 2022, no. 3, pp. 78-85, 2022.

[61] N. H. Pham, “Research on SSH brute-force prevention solutions for government information systems,” *Master’s Thesis, Academy of Cryptography Techniques*, 2020.

[62] D. K. Tran and T. T. H. Nguyen, “Application of Isolation Forest in anomaly detection on system log data,” *Journal of Scientific Research and Development*, vol. 2, no. 4, pp. 112-121, 2023.

[63] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305-316.

Chapter 4

CHAPTER 3: METHODOLOGY

4.1 3.1 Research Design

4.1.1 3.1.1 Overall System Architecture

The SSH brute-force attack detection and prevention system is designed following a microservices architecture, deployed on Docker with a total of 9 services operating in coordination. This architecture ensures scalability, maintainability, and flexible deployment in real-world environments.

[Figure 3.1: Overall system architecture for SSH brute-force attack detection]

The principal components of the system are organized into five functional layers:

Data Collection Layer. This layer is responsible for collecting authentication logs from SSH servers through log tailing and event forwarding mechanisms. Log data is standardized and ingested into the ELK Stack (Elasticsearch, Logstash, Kibana) for storage and indexing.

Processing and Analysis Layer. The central layer of the system comprises the data preprocessing module, the feature extraction module, and the AI model inference module. These modules are implemented as RESTful APIs using the FastAPI framework, enabling real-time request processing with high performance.

Decision Layer. This layer implements the dynamic threshold algorithm based on the EWMA-Adaptive Percentile method, combining anomaly scores from the AI models to render classification decisions: normal or attack.

Response Layer. Fail2Ban is integrated to automatically execute prevention measures upon attack detection, including IP address banning, email or webhook alert notifications, and event logging to the monitoring system.

Presentation Layer. A web interface developed in React provides a real-time monitoring dashboard displaying detection events, attack statistics, and administrative configuration controls.

[Table 3.1: List of 9 Docker services in the system]

No.	Service	Technology	Function
1	API Server	FastAPI (Python)	Business logic, model inference
2	Frontend	React	Monitoring and administration interface
3	Elasticsearch	ELK Stack	Log storage and indexing
4	Logstash	ELK Stack	Log collection and normalization
5	Kibana	ELK Stack	Data visualization
6	Fail2Ban	Fail2Ban	Automated IP blocking
7	Redis	Redis	Cache and message queue
8	Database	PostgreSQL	Configuration and results storage
9	Nginx	Nginx	Reverse proxy and load balancing

[Figure 3.2: Data flow pipeline of the system]

The data processing pipeline operates as follows: (1) SSH authentication logs from the server are collected by Logstash; (2) Logstash parses and forwards the normalized logs to Elasticsearch; (3) the API Server periodically queries Elasticsearch or receives events via webhook to obtain new log data; (4) the preprocessing module performs feature extraction over the sliding time window; (5) the AI model computes anomaly scores; (6) the dynamic threshold algorithm compares the anomaly score against the current threshold; (7) if an attack is detected, the system triggers Fail2Ban and dispatches alerts; (8) results are displayed on the React dashboard.

4.1.2 3.1.2 Semi-Supervised Anomaly Detection Approach

The research adopts the semi-supervised anomaly detection paradigm, in which models are trained exclusively on normal data and subsequently identify patterns that deviate from the normal distribution as anomalies. This approach was selected for three reasons:

First, **practicality.** In practice, normal data is far easier to collect than labeled attack data. Organizations can readily collect logs of normal operations but cannot feasibly obtain samples of every possible attack type.

Second, **detection of novel attacks.** Semi-supervised models can detect attack types never previously encountered (zero-day attacks), provided that the attack behavior deviates from the learned normal patterns.

Third, **avoidance of class imbalance issues.** Supervised methods frequently suffer from class imbalance when the proportion of attacks is low. The semi-supervised approach is unaffected by this issue.

4.2 3.2 Data Collection Methods

4.2.1 3.2.1 Data Sources

The research employs two primary data sources to construct training and test datasets. The dual-source approach is a deliberate methodological choice that addresses the fundamental data challenge in anomaly detection: obtaining both clean normal data (for training) and diverse attack data (for evaluation). By combining a controlled simulation environment (which provides guaranteed-clean normal data) with a honeypot deployed on the public Internet (which provides authentic attack data), the study achieves a dataset that is both methodologically sound and practically representative.

The two data sources are described in detail below:

Source 1 — Honeypot data (honeypot_auth.log). This authentication log was collected from an SSH honeypot server deployed on the public Internet. A honeypot is a system specifically designed to attract and record real-world attack activity, providing highly representative attack data.

- **Log lines:** 119,729
- **Collection duration:** 5 continuous days
- **Unique IP addresses:** 679
- **Failed password events:** 29,301
- **Accepted root logins (from admin IPs):** 532 from 6 verified administrative IP addresses
- **Hostname:** "mail"

The honeypot server was configured with SSH open on the default port (port 22), using the hostname "mail" to simulate a real email server and attract more brute-force attacks. During operation, only 6 administrative IP addresses were confirmed as legitimate; all other connections were classified as attack activity.

Source 2 — Simulation data (simulation_auth.log). This authentication log was generated from a controlled simulation environment representing normal daily SSH activity within an organization.

- **Log lines:** 54,521
- **User accounts:** 64
- **Successful logins (Accepted):** 4,205
- **Failed authentication events (Failed):** 177
- **Hostname:** "if"

The simulation data was designed to reflect the behavioral patterns of legitimate users, including: logins during business hours, logins from multiple devices, accidental password mistyping, and sessions of varying duration.

[Table 3.2: Summary of two data sources]

Attribute	honeypot_auth.log	simulation_auth.log
Log lines	119,729	54,521
Collection duration	5 days	Continuous
Unique IPs	679	–
User accounts	–	64
Successful logins	532 (root)	4,205
Failed logins	29,301	177
Hostname	"mail"	"if"
Data nature	Predominantly attack	Entirely normal

4.2.2 3.2.2 Log Data Format

SSH log data conforms to the standard Linux syslog format, recorded in /var/log/auth.log. Each log line contains: timestamp, hostname, service name, process PID, and message content. The principal event types include:

- **Failed password:** Failed password authentication, containing username, source IP, and connection port.
- **Accepted password / Accepted publickey:** Successful authentication with analogous information.
- **Invalid user:** The attempted username does not exist on the system.
- **Connection closed / Connection reset:** Connection termination events.
- **PAM authentication failure:** Pluggable Authentication Module failure.
- **Maximum authentication attempts exceeded:** The per-session authentication limit was reached.

4.2.3 3.2.3 Labeling Strategy

Given the semi-supervised anomaly detection paradigm, the labeling strategy was carefully designed to ensure accuracy and compatibility with the training methodology:

For simulation data (simulation_auth.log): All data is labeled as **normal**. The simulation environment is fully controlled, and all activities are performed by legitimate users, including accidental password mistyping. This accurately reflects the reality that occasional password errors constitute normal user behavior.

For honeypot data (honeypot_auth.log): The labeling strategy is based on the list of 6 verified administrative IP addresses: - Successful root login events (Accepted root) from the 6 admin IPs are labeled **normal**. - All remaining events (including failed passwords, invalid users, and logins from unverified IPs) are labeled **attack**.

4.2.4 3.2.4 Data Splitting

Data is split following the semi-supervised learning paradigm:

Training set: - Size: **7,212 samples** - Composition: **100% normal** (exclusively normal samples) - Source: 70% of simulation_auth.log data - Purpose: Train models to learn normal behavior characteristics

Test set: - Size: **15,184 samples** - Composition: 3,796 normal + 11,388 attack samples - Normal-to-attack ratio: **1:3** - Source: Remaining 30% of simulation_auth.log (normal) + honeypot_auth.log data (attack and normal from admin IPs)

[Table 3.3: Dataset split summary]

Dataset	Samples	Normal	Attack	Ratio
Training	7,212	7,212 (100%)	0 (0%)	–
Test	15,184	3,796 (25%)	11,388 (75%)	1:3
Total	22,396	11,008	11,388	–

4.3 3.3 Sampling Data Analysis Techniques

4.3.1 3.3.1 Feature Extraction via Sliding Window

Features are extracted using the sliding window method with the following parameters:

- **Window size:** 5 minutes
- **Stride:** 1 minute
- **Aggregation unit:** Per source IP address

Each 5-minute window for each IP address produces a 14-dimensional feature vector. The 5-minute window size was selected based on the following observations: it is sufficiently short to enable early detection of rapid brute-force attacks; it is sufficiently long to accumulate statistically meaningful behavioral information; and the 1-minute stride ensures high temporal resolution, allowing detection with a maximum delay of 1 minute.

[Figure 3.3: Illustration of the sliding window method (window=5 min, stride=1 min)]

4.3.2 3.3.2 Description of the 14 Features

The research designs 14 features reflecting different aspects of SSH authentication behavior within each time window, organized into five functional groups:

Group 1: Authentication Count and Rate Features

1. fail_count (Number of failed authentication attempts). The total number of failed password events from a given IP within the time window. This is the most fundamental and direct feature for identifying brute-force attacks, as attackers typically generate a large volume of incorrect password attempts.

2. success_count (Number of successful authentication events). The total number of successful authentication events. Legitimate users typically have a high success rate, while brute-force attackers rarely succeed.

3. fail_rate (Failure rate). The ratio of failed attempts to total authentication attempts: fail_rate = fail_count / (fail_count + success_count). Values approaching 1.0 indicate that nearly all attempts failed, characteristic of brute-force attacks. Normal users typically have fail_rate below 0.3.

Group 2: Username-Related Features

4. unique_usernames (Number of distinct usernames). The number of different usernames used in authentication attempts. Credential stuffing and dictionary attacks typically try many different usernames (root, admin, test, oracle, etc.), while legitimate users typically use only 1-2 accounts.

5. invalid_user_count (Number of invalid user attempts). The total number of login attempts with usernames that do not exist on the system. This is a clear indicator of an attack, as legitimate users know their own account names.

6. invalid_user_ratio (Invalid user ratio). The ratio of invalid user attempts to total authentication attempts. A high ratio indicates that the attacker is scanning the system with random or common username lists.

Group 3: Connection and Temporal Features

7. connection_count (Total number of connections). The total number of SSH connections (both successful and failed) from a given IP. Automated brute-force attacks typically generate a very large number of connections in a short period.

8. mean_inter_attempt_time (Mean time between attempts, in seconds). The average interval between consecutive authentication attempts. Automated attack tools typically have very short and uniform intervals (often under 1 second), while manual human users have longer and more variable intervals.

9. std_inter_attempt_time (Standard deviation of inter-attempt time, in seconds). A low standard deviation indicates a uniform behavioral pattern, characteristic of automated tools. A high standard deviation indicates irregular behavior, more likely from a manual human user.

10. min_inter_attempt_time (Minimum time between attempts, in seconds). The smallest interval between any two consecutive attempts. A value near zero indicates that at least one pair of attempts occurred nearly simultaneously, a hallmark of automated attacks.

Group 4: Network and Session Features

11. unique_ports (Number of distinct source ports). The number of different source ports used in connections from a given IP. Each new TCP connection typically uses a different random source port. A large number of unique ports correlates with a large number of distinct connections, reflecting the activity level of the IP.

12. session_duration_mean (Mean session duration, in seconds). The average duration of SSH sessions.

Legitimate users typically have sessions lasting minutes to hours, while brute-force attacks produce very short sessions (under a few seconds), because each failed password attempt results in rapid disconnection.

Group 5: Attack Indicator Features

13. pam_failure_escalation (PAM failure escalation). A binary variable (0 or 1) indicating whether a continuously escalating sequence of PAM failures occurred within the time window. A value of 1 indicates systematic brute-force attack behavior with progressively increasing PAM failures.

14. max_retries_exceeded (Maximum retries exceeded). A binary variable indicating whether a "maximum authentication attempts exceeded" event occurred within the time window. This is a direct indicator of brute-force attacks, when the attacker attempts multiple passwords within a single connection session until the SSH server terminates the connection.

[Table 3.4: Summary of 14 features and their significance]

No.	Feature	Data Type	Detection Significance
1	fail_count	Integer	Number of incorrect password attempts
2	success_count	Integer	Number of successful logins
3	fail_rate	Float [0,1]	Failure-to-total ratio
4	unique_usernames	Integer	Diversity of attempted usernames
5	invalid_user_count	Integer	Non-existent username attempts
6	invalid_user_ratio	Float [0,1]	Proportion of invalid usernames
7	connection_count	Integer	Total SSH connections
8	mean_inter_attempt_time	Float (seconds)	Average speed between attempts
9	std_inter_attempt_time	Float (seconds)	Variability of attempt speed
10	min_inter_attempt_time	Float (seconds)	Fastest speed between attempts
11	unique_ports	Integer	Diversity of source ports
12	session_duration_mean	Float (seconds)	Average session length
13	pam_failure_escalation	Binary {0,1}	Escalating PAM failure sequence
14	max_retries_exceeded	Binary {0,1}	Maximum retry limit reached

4.3.3 3.3.3 Temporal Feature Engineering Rationale

The design of the 14 features reflects a deliberate balance between comprehensiveness and computational efficiency. The features were selected to capture the four fundamental dimensions of SSH authentication behavior that differ between normal users and automated attack tools: the volume dimension (how many attempts are made), the diversity dimension (how varied the attempts are), the temporal dimension (how fast the attempts occur), and the session dimension (how long the connections last).

The temporal features (mean_inter_attempt_time, std_inter_attempt_time, min_inter_attempt_time) deserve particular attention because they capture the most fundamental difference between human and automated behavior. Human users interact with SSH servers at human timescales: typing a password takes several seconds, deciding to retry after a failure takes additional seconds, and the variability between attempts is high because human cognition is inherently inconsistent. Automated attack tools, in contrast, operate at machine timescales: password attempts are dispatched in milliseconds, the intervals between attempts are uniform (determined by network latency rather than human thought), and there is virtually no variability unless the tool is specifically configured to introduce randomization.

The session_duration_mean feature captures a related but distinct aspect of behavior. When a legitimate user connects via SSH, the session typically lasts from several minutes (for a quick administrative task) to several hours (for a development or deployment session). A brute-force attack, however, produces sessions of only a few seconds: the attacker connects, the authentication fails, the server terminates the connection, and the attacker immediately reconnects to try again. This extremely short session duration is a reliable indicator of automated attack behavior that is difficult for attackers to disguise, because the session duration is determined by the server's response time rather than the attacker's configuration.

The attack indicator features (pam_failure_escalation, max_retries_exceeded) provide binary signals that complement the continuous features. These features are based on specific events in the SSH authentication log that are almost exclusively associated with attack activity: an escalating sequence of PAM failures indicates a systematic brute-force attempt, and exceeding the maximum authentication retry limit indicates that the attacker is exhausting all permitted attempts within a single connection before reconnecting.

4.3.4 3.3.4 Feature Scaling

After extraction, features are normalized using **RobustScaler** from the scikit-learn library. RobustScaler was selected over StandardScaler or MinMaxScaler for the following reasons:

First, **robustness to outliers.** RobustScaler uses the median and interquartile range (IQR) rather than the mean and standard deviation, reducing the influence of outlier values in the data.

Second, **suitability for attack data.** Brute-force attack data frequently contains extreme values (e.g., fail_count can reach thousands while the normal value is 0-2). RobustScaler is unaffected by these extreme values.

The RobustScaler normalization formula is:

$$x_{scaled} = \frac{x - Q_2(x)}{Q_3(x) - Q_1(x)}$$

where Q1, Q2, and Q3 are the 25th, 50th (median), and 75th percentiles of the feature x, respectively. The scaler is fit on the training set (containing only normal data) and applied to transform both the training and test sets,

ensuring no data leakage.

4.3.5 3.3.4 Model Selection and Configuration

Three anomaly detection models are selected and compared:

Isolation Forest (IF). Based on the isolation principle, where anomalous points are isolated more rapidly than normal points in random decision trees. The algorithm constructs an ensemble of isolation trees with random feature splits. The anomaly score is inversely proportional to the average path length from root to leaf. Advantages: high computational efficiency (O(n log n) complexity), suitability for high-dimensional data, and no distributional assumptions.

Optimized hyperparameters (baseline): n_estimators=300, max_samples=512, max_features=0.5 (7 of 14 features per tree).

Optimized hyperparameters (optimized configuration): contamination=0.01, max_features=0.75, max_samples=512, n_estimators=500.

Local Outlier Factor (LOF). Based on local density comparison, comparing the local density of each point with its k-nearest neighbors. Points with significantly lower local density than their neighbors are classified as anomalies. Hyperparameters: n_neighbors=30, novelty=True, metric=Minkowski.

One-Class SVM (OCSVM). Extends SVM for one-class classification by finding a maximum-margin hyperplane in the kernel-mapped feature space that separates training data from the origin. Hyperparameters: kernel=RBF, gamma=auto (1/14 approximately 0.0714), nu=0.01.

[Table 3.5: Comparison of characteristics of three models]

Characteristic	Isolation Forest	LOF	OCSVM
Principle	Isolation	Local density	Maximum-margin hyperplane
Complexity	O(n log n)	O(n^2)	O(n^2 to n^3)
Large data suitability	Excellent	Moderate	Moderate
Local anomaly detection	Moderate	Excellent	Good
Distributional assumptions	None	None	None (with kernel)
Interpretability	Moderate	Low	Low

4.3.6 3.3.5 Dynamic Threshold Algorithm

The EWMA-Adaptive Percentile dynamic threshold algorithm is designed to address the limitations of static thresholds. The algorithm parameters are:

- alpha = 0.3: EWMA smoothing factor

- base_percentile = 95: Base percentile
- sensitivity_factor = 1.5: Sensitivity factor
- lookback = 100: Number of recent data points for computation

Step 1: Compute EWMA of anomaly scores. Given the anomaly score time series s_1, s_2, …, s_t, the EWMA at time t is:

$$\mu_t^{EWMA} = \alpha \cdot s_t + (1 - \alpha) \cdot \mu_{t-1}^{EWMA}$$

with initialization mu_0 = s_1. The alpha=0.3 value ensures the EWMA responds quickly to recent changes while remaining robust to transient noise.

Step 2: Compute the adaptive percentile. Using a lookback window of L=100 most recent anomaly scores, compute the 95th percentile:

$$P_t = Percentile(\{s_{t-L+1}, ..., s_t\}, 95)$$

Step 3: Compute the dynamic threshold.

$$\theta_t = \mu_t^{EWMA} + sensitivity_factor \times (P_t - \mu_t^{EWMA})$$

$$\theta_t = \mu_t^{EWMA} + 1.5 \times (P_t - \mu_t^{EWMA})$$

This formula places the threshold at the EWMA mean plus 1.5 times the distance from the EWMA to the 95th percentile, ensuring: the threshold lies above the vast majority (>95%) of normal scores when the distribution is stable; the EWMA rises during attack waves, pulling the threshold upward and preventing continuous false alarms; and the EWMA decreases after attacks subside, returning the threshold to normal levels.

Step 4: Classification decision. A sample at time t is classified as attack if s_t > theta_t, and normal otherwise.

[Figure 3.5: Illustration of the EWMA-Adaptive Percentile dynamic threshold algorithm]

[Table 3.6: Dynamic threshold algorithm parameters and their meanings]

Parameter	Value	Meaning	Effect
alpha	0.3	EWMA response speed	Large: fast response, more noise; Small: slow response, stable

Parameter	Value	Meaning	Effect
base_percentile	95	Base percentile threshold	High: fewer FP, may miss attacks; Low: more FP, better detection
sensitivity_factor	1.5	Detection sensitivity	High: higher threshold, fewer alerts; Low: lower threshold, more alerts
lookback	100	Recent sample count	Large: stable, slow adaptation; Small: responsive, fast adaptation

4.3.7 3.3.6 Training Process

The training process follows the semi-supervised novelty detection paradigm in four sequential steps:

Step 1 — Data preparation. Only the 7,212 normal samples from 70% of the simulation data are used for training. This ensures that the models learn exclusively from legitimate behavior patterns, with no contamination from attack data.

Step 2 — Normalization. The RobustScaler is fit on the training set, and the transformation is applied to both the training and test sets. This approach prevents data leakage: the scaler parameters are derived solely from the training data, and the same transformation is applied consistently to all data.

Step 3 — Model training. Each of the three models (IF, LOF, OCSVM) is fit on the normalized training set. The Isolation Forest constructs 300 isolation trees (baseline) or 500 trees (optimized), each using 512 sub-samples and 50% (baseline) or 75% (optimized) of the 14 features. The LOF model computes the local reachability density for each training point using 30 nearest neighbors. The OCSVM model solves the quadratic optimization problem to find the maximum-margin hyperplane in the RBF kernel space.

Step 4 — Prediction and evaluation. Anomaly scores are computed for each of the 15,184 test samples. The scores are then compared against the classification threshold (either static or dynamic) to produce binary predictions (normal/attack). Performance is evaluated using the five metrics defined in Section 3.3.5 (Accuracy, Precision, Recall, F1-Score, ROC-AUC).

The hyperparameter optimization process uses grid search combined with cross-validation on the training set. Since the training set contains only normal data, the optimization criterion is based on the model's ability to

reconstruct the normal data distribution and the ROC-AUC score on a small validation set containing a subset of attack samples.

For the Isolation Forest, the grid search explores: n_estimators in {100, 200, 300, 500}, max_samples in {256, 512, 1024}, max_features in {0.25, 0.5, 0.75, 1.0}, and contamination in {auto, 0.01, 0.05, 0.1}. The use of max_features=0.5 (baseline) or 0.75 (optimized) increases diversity among the trees in the ensemble, reducing the risk of overfitting. The max_samples=512 parameter limits the sub-sample size per tree, which accelerates training and improves anomaly detection according to Isolation Forest theory (anomalous points are more easily isolated in smaller samples). The n_estimators=300 (baseline) or 500 (optimized) ensures sufficient tree count to stabilize the anomaly score.

For LOF, the n_neighbors=30 value balances between local anomaly detection sensitivity (lower k) and estimation stability (higher k). The novelty=True setting enables the model to predict on new data points not seen during training. For OCSVM, the nu=0.01 parameter sets the upper bound on the outlier fraction at 1%, consistent with the assumption that the training set is purely normal and allows at most 1% of samples to be misclassified. The gamma=auto setting automatically computes gamma = 1/n_features = 1/14, approximately 0.0714.

4.4 3.4 Limitations of the Methodology

4.4.1 3.4.1 Real-Time Detection Pipeline

The real-time detection pipeline is designed for continuous processing of SSH log event streams with minimal latency. The pipeline comprises five stages:

The pipeline is designed to process SSH log events as they arrive with minimal latency, ensuring that detection and response occur within seconds of the original authentication event. Each stage is optimized for its specific computational requirements, with I/O-bound stages using asynchronous processing and CPU-bound stages using parallel execution.

Stage 1 — Ingestion. Logstash tails the auth.log file on SSH servers, parses each new log line, and sends structured events to Elasticsearch. Simultaneously, a webhook/event stream forwards events to the API Server.

Stage 2 — Aggregation. The API Server maintains per-IP sliding window buffers. At each 1-minute stride, the system aggregates events from the most recent 5-minute window to compute the 14 features.

Stage 3 — Scoring. The feature vector is normalized using the pre-fitted RobustScaler and fed into the AI model to compute the anomaly score. All three models (IF, LOF, OCSVM) can operate in parallel or in an ensemble configuration.

Stage 4 — Decision. The anomaly score is compared against the current dynamic threshold theta_t. Simultaneously, the EWMA-Adaptive Percentile algorithm updates the threshold based on the newly received score.

Stage 5 — Action. If an attack is detected, the system triggers prevention actions.

[Figure 3.6: Five-stage real-time detection pipeline]

To ensure real-time performance, the pipeline employs: asynchronous processing (async/await) via FastAPI for I/O-bound tasks such as Elasticsearch queries and database writes; parallel processing through thread pools or process pools for CPU-bound tasks such as model computation; and Redis caching for intermediate results including scaler parameters, model objects, and per-IP sliding window states.

4.4.2 3.4.2 Fail2Ban Integration and Alert System

When the detection module identifies an IP as conducting a brute-force attack, the system invokes the Fail2Ban API to: (1) ban the IP address, blocking all new connections from that IP; (2) apply a configurable ban duration based on the severity of the attack; and (3) implement progressive banning, where repeated offenders receive exponentially increasing ban durations.

The multi-channel alert system supports: real-time alert display on the React dashboard with IP information, timestamp, anomaly score, predicted attack type, and action taken; webhook notifications to external services (Slack, Telegram, etc.); and comprehensive logging of all detection events and response actions to Elasticsearch for post-incident analysis.

An administrator feedback mechanism allows security personnel to mark alerts as true positive or false positive through the React interface. This feedback is stored and can be used for future model refinement or threshold parameter adjustment.

4.4.3 3.4.3 Evaluation Metrics

Performance evaluation is based on the standard binary classification confusion matrix (normal vs. attack):

Accuracy = (TP + TN) / (TP + TN + FP + FN), measuring the overall correct classification rate. In imbalanced scenarios, accuracy may not accurately reflect model performance.

Precision = TP / (TP + FP), measuring the proportion of predicted attacks that are actual attacks. High precision indicates few false alarms.

Recall (Sensitivity) = TP / (TP + FN), measuring the proportion of actual attacks that are detected. In cybersecurity, recall is particularly critical because missing an attack (false negative) can have severe consequences.

F1-Score = 2 * (Precision * Recall) / (Precision + Recall), the harmonic mean of Precision and Recall, providing a balanced measure.

ROC-AUC, the area under the Receiver Operating Characteristic curve, measures the model's ability to discriminate between normal and attack classes across all classification thresholds. ROC-AUC=1.0 is ideal; ROC-AUC=0.5 is equivalent to random classification.

[Table 3.7: Summary of evaluation metrics and their security significance]

Metric	Security Significance	Priority
Accuracy	Overall performance	Medium
Precision	Reducing false alarms (alert fatigue)	High
Recall	Not missing attacks	Very High
F1-Score	Precision-Recall balance	High
ROC-AUC	General discriminative ability	High

Additionally, system-level performance metrics are evaluated: latency (time from log event receipt to classification decision), throughput (events processed per unit time), memory usage, and CPU usage. These metrics ensure the system can operate under real-world production requirements.

4.4.4 3.4.4 Experimental Design for Attack Scenario Evaluation

To evaluate the system comprehensively, five attack scenarios were designed to cover the major categories of SSH brute-force attacks observed in the wild. Each scenario is defined by a specific set of parameters that control the attack behavior, including the rate of attempts, the number of source IP addresses, the username selection strategy, and the password selection strategy.

Scenario 1: Basic Brute-Force. A single IP address attempts to authenticate as root (or another target account) at the maximum possible rate. The attacker uses a systematically generated list of passwords, sending attempts as fast as the network and SSH server will permit. This scenario produces the most extreme feature values and serves as the baseline for evaluating detection capability. The expected detection rate is 100% for all models.

Scenario 2: Distributed Attack. Multiple IP addresses (simulating a botnet) each attempt a small number of authentication attempts, distributing the total attack volume across many sources. Each IP may attempt only 2-5 passwords before moving on to the next target server, or cycling back at a later time. The challenge for the detection system is that per-IP feature values may fall within the range of normal behavior. Detection relies on the temporal and session features that remain anomalous even at low per-IP volumes, because automated tools still produce characteristically short sessions and machine-speed timing patterns.

Scenario 3: Low-and-Slow. A single IP address spaces its authentication attempts over long intervals (30-120 seconds between attempts), deliberately staying below traditional rate-limiting thresholds. This is the most sophisticated evasion technique in the brute-force attacker's repertoire, and it represents the greatest challenge for any detection system based on short-term behavioral analysis. The EWMA accumulation mechanism is specifically designed to address this scenario by detecting the cumulative build-up of anomalous activity over multiple detection windows.

Scenario 4: Credential Stuffing. The attacker uses a database of username-password pairs leaked from breaches at other services, testing each pair against the target SSH server. The distinguishing characteristic of this attack is the large number of unique usernames, many of which do not exist on the target system. The unique_usernames and invalid_user_ratio features are expected to be the primary discriminators.

Scenario 5: Dictionary Attack. A single IP address targets a specific account (typically root) using a dictionary of common passwords. This scenario is similar to basic brute-force but more realistic, as real-world attackers typically use curated password lists rather than exhaustive enumeration. The attack proceeds at high speed with a single username.

The scenarios were implemented as Python scripts using the Paramiko SSH library, which provides programmatic control over SSH client connections and enables precise manipulation of timing, username selection, and password selection parameters. Each scenario was executed against the test environment and the resulting SSH log data was processed through the complete detection pipeline.

4.4.5 3.4.5 Ethical Considerations

The data collection methodology raises several ethical considerations that were addressed in the research design. The honeypot data was collected from a purpose-built honeypot server specifically designed to attract and record attack traffic. No legitimate user data was collected or compromised during the honeypot operation. The 6 administrative IP addresses that produced legitimate traffic on the honeypot were under the full control of the research team, and all data was handled in accordance with responsible research practices.

The simulation data was generated synthetically, with no real user data involved. The 64 simulated user accounts and their behavioral patterns were designed to represent realistic but entirely fictional usage scenarios. No actual user credentials, IP addresses, or session data from real individuals were used in the simulation.

The attack scenario testing was conducted entirely within a controlled test environment isolated from production systems. No unauthorized access attempts were made against any external systems. The attack simulation scripts are documented for reproducibility but are not distributed publicly in executable form to prevent misuse.

4.4.6 3.4.6 Methodological Constraints

The methodology has several inherent constraints that should be noted. The 5-minute sliding window, while effective for detecting rapid and moderate-speed attacks, may be insufficient for detecting extremely slow attacks where the attacker spaces attempts over intervals exceeding the window size. This limitation suggests the need for supplementary long-term IP profiling or integration with threat intelligence feeds.

The semi-supervised training approach assumes that the training data is purely normal. Any contamination of the training set with attack samples could degrade model performance. The labeling strategy was designed to minimize this risk, but absolute purity cannot be guaranteed in all deployment scenarios.

The RobustScaler normalization is fit on the training data, which represents a single operational environment. When deploying to a new environment with different usage patterns, the scaler and model should be retrained on data representative of that environment.

Finally, the evaluation is conducted on a specific dataset with a fixed normal-to-attack ratio of 1:3. Performance may vary under different ratios or in environments with significantly different traffic patterns. The dynamic threshold mechanism is designed to mitigate this sensitivity, but comprehensive evaluation across diverse environments remains an area for future work.

Chapter 5

CHAPTER 4: EXPERIMENTAL AND RESULTS

5.1 4.1 Introduction

This chapter presents the comprehensive experimental results of the SSH brute-force attack detection and prevention system. The experiments are organized in a logical progression, beginning with exploratory data analysis of the raw datasets, proceeding through feature analysis and model training, and culminating in dynamic threshold evaluation and attack scenario testing. The experiments were designed to evaluate the system across multiple dimensions: the quality and characteristics of the dataset, the discriminative power of the engineered features, the comparative performance of the three anomaly detection models (Isolation Forest, LOF, One-Class SVM), the effectiveness of the EWMA-Adaptive Percentile dynamic threshold, and the system's ability to detect five distinct attack scenarios in real-time. All experiments were conducted using Python with the scikit-learn library on the Docker-containerized platform comprising 9 services.

The dataset, after collection, preprocessing, and feature extraction via the 5-minute sliding window with 1-minute stride, yielded a total of 22,396 samples. The training set consists of 7,212 exclusively normal samples, while the test set comprises 15,184 samples (3,796 normal + 11,388 attack), reflecting a normal-to-attack ratio of approximately 1:3. All models were implemented using scikit-learn version 1.x, and feature extraction was performed using pandas and numpy. The dynamic threshold was implemented as a custom Python module with configurable parameters for alpha, base_percentile, sensitivity_factor, and lookback window size. The complete pipeline was orchestrated through the FastAPI server, which manages the data flow from Elasticsearch queries through feature extraction, model inference, and threshold comparison to the final classification decision and response action.

5.2 4.2 Presentation of Data

5.2.1 4.2.1 Overall Dataset Statistics

[Table 4.1: Overall dataset statistics]

Attribute	Value
Total samples	22,396
Training set	7,212 samples (100% normal)
Test set	15,184 samples (3,796 normal + 11,388 attack)
Number of features	14
Window size	5 minutes
Stride	1 minute

5.2.2 4.2.2 Honeypot Data Analysis

The honeypot_auth.log dataset (119,729 log lines over 5 days) provides a comprehensive picture of SSH brute-force attack activity on the public Internet. Exploratory data analysis (EDA) revealed several important characteristics:

Temporal distribution. Attacks occurred continuously 24/7 with no discernible time-of-day pattern, indicating the use of automated attack tools. Over the 5-day collection period, an average of approximately 5,860 failed password attempts were recorded per day (29,301 total over 5 days).

IP distribution. A total of 679 unique IP addresses were recorded. The distribution is strongly right-skewed: a small number of IPs generated thousands of attempts, while the majority of IPs generated only tens to hundreds. The top 10% of IPs contributed more than 60% of total failed attempts.

[Figure 4.1: Distribution of failed authentication attempts per IP (honeypot)]

Username targeting analysis. The most frequently targeted username was “root,” which accounted for an estimated 40% or more of all attempts. This is consistent with the well-documented tendency of SSH brute-force attacks to prioritize the root account, which provides unrestricted system access upon compromise. The remaining targeted usernames followed a predictable distribution: “admin,” “test,” “user,” “oracle,” “postgres,” “ubuntu,” “ftp,” “mysql,” and other service account names that are commonly present on Linux servers. The diversity of usernames attempted reveals the attacker strategy of combining targeted high-value accounts with broad enumeration of common service accounts, reflecting a sophisticated understanding of typical server configurations. Of the 119,729 total log lines, 29,301 were failed password events.

Legitimate administrative activity. The 6 verified administrative IP addresses generated 532 successful root login sessions, distributed primarily during business hours, with session durations ranging from several minutes

to several hours — characteristic of normal system administration activity.

[Table 4.2: Detailed honeypot data statistics]

Statistic	Value
Total log lines	119,729
Collection duration	5 days
Unique IPs	679
Failed password events	29,301
Accepted root (admin IPs)	532
Number of admin IPs	6
Average failed/day	~5,860
Average failed/IP	~43.2

5.2.3 4.2.3 Simulation Data Analysis

The simulation_auth.log dataset (54,521 log lines, 64 user accounts) reflects normal SSH activity patterns within an organization:

Temporal distribution. Activity was concentrated primarily during business hours (8:00-18:00), with a marked decrease during nighttime and weekends, accurately reflecting real-world work patterns.

High success rate. With 4,205 successful logins and only 177 failures, the success rate reached approximately 95.96% (4,205/(4,205+177)). The failure rate of only 4.04% reflects cases of accidental password mistyping — entirely normal behavior.

User distribution. The 64 user accounts exhibited varying levels of SSH usage. Some accounts (e.g., developers, system administrators) had significantly higher login frequencies than ordinary users.

[Figure 4.2: Comparison of hourly activity distribution between honeypot (attack) and simulation (normal)]

[Table 4.3: Detailed simulation data statistics]

Statistic	Value
Total log lines	54,521
User accounts	64
Accepted logins	4,205
Failed logins	177
Success rate	95.96%
Failure rate	4.04%

5.2.4 4.2.4 Feature Comparison Between Normal and Attack Classes

Exploratory analysis revealed clear differences between normal and attack behavior across nearly all 14 features:

[Table 4.4: Descriptive statistics of key features by label]

Feature	Normal (mean +/- std)	Attack (mean +/- std)	Difference
fail_count	Low (0-2)	High (tens to hundreds)	Very large
fail_rate	< 0.3	> 0.9	Very large
unique_usernames	1-2	5-20+	Large
invalid_user_count	~0	High	Very large
mean_inter_attempt_time	High (>10s)	Very low (<1s)	Very large
session_duration_mean	High (minutes-hours)	Very low (<5s)	Very large
connection_count	Low (1-5)	High (tens to hundreds)	Large

The analysis confirmed that temporal features (mean_inter_attempt_time, min_inter_attempt_time, session_duration_mean) and ratio features (fail_rate, invalid_user_ratio) exhibit the greatest discriminative power between the two classes. This is consistent with the intuition that automated brute-force attacks generate temporal patterns fundamentally different from manual human activity.

[Figure 4.3: Boxplot comparison of key feature distributions between Normal and Attack classes]

5.2.5 4.2.5 Statistical Summary of the Combined Dataset

The combined dataset, after feature extraction via the 5-minute sliding window approach, presents a rich and challenging classification problem. The training set of 7,212 samples represents the full range of normal SSH behavior observed in the simulation environment, including peak-hour login clusters, off-hour maintenance sessions, and accidental password failures. The test set of 15,184 samples (3,796 normal + 11,388 attack) creates a realistic imbalanced classification scenario with a 1:3 normal-to-attack ratio, reflecting the high proportion of attack traffic that public-facing SSH servers typically experience.

The class imbalance in the test set is deliberate and mirrors real-world conditions. On the honeypot server, attack traffic vastly outnumbered legitimate traffic (29,301 failed password events versus 532 legitimate root logins). In production environments, while the absolute ratio varies, the general pattern of attack traffic exceeding normal traffic on public-facing SSH servers is well documented in the literature. The 1:3 ratio used in this study represents a conservative estimate that avoids the extreme imbalance that would make evaluation metrics less informative.

The choice to use exclusively normal data for training (7,212 samples, 100% normal) is a defining characteristic of the semi-supervised approach. This design reflects the practical reality that organizations typically have

ready access to logs of normal operations but lack comprehensive labeled attack datasets. The training set must be large enough to capture the full variability of normal behavior — including rare but legitimate patterns such as late-night maintenance sessions or multi-device login sequences — while remaining free of any attack contamination that could corrupt the learned normal profile.

5.3 4.3 Analysis of Results

5.3.1 4.3.1 Feature Distribution Analysis

Distribution analysis of the 14 features across the dataset reveals distinct patterns:

Count features (fail_count, success_count, connection_count, invalid_user_count) exhibit strongly right-skewed distributions with long tails. The majority of normal samples have values near 0, while attack samples spread broadly from low to very high values. This explains why RobustScaler is more appropriate than StandardScaler for this data.

Ratio features (fail_rate, invalid_user_ratio) display clearly bimodal distributions. Normal samples cluster around 0 (few failures), while attack samples cluster around 1 (nearly all failures). The gap between the two modes creates a natural classification boundary.

Temporal features (mean_inter_attempt_time, std_inter_attempt_time, min_inter_attempt_time) have more complex distributions. Normal samples have broadly dispersed values (reflecting the irregularity of manual behavior), while attack samples concentrate near 0 (reflecting the high speed of automated attacks).

Binary features (pam_failure_escalation, max_retries_exceeded) have binary distributions. In normal samples, the vast majority have value 0; in attack samples, the proportion with value 1 is significantly higher.

[Figure 4.4: Histogram distribution of 14 features, separated by normal/attack labels]

5.3.2 4.3.2 Correlation Analysis

The Pearson correlation matrix among the 14 features reveals several highly correlated pairs:

- **fail_count and connection_count:** Strong positive correlation ($r > 0.8$), as each password attempt creates a connection.
- **fail_count and fail_rate:** High positive correlation, particularly when success_count is low.
- **mean_inter_attempt_time and std_inter_attempt_time:** Moderate positive correlation (~0.5-0.7), as both reflect temporal aspects.
- **invalid_user_count and invalid_user_ratio:** High positive correlation.
- **unique_usernames and invalid_user_count:** Positive correlation, as attacks typically use many invalid usernames.

Despite some features exhibiting high correlation, all 14 features were retained because: (1) the anomaly detection models used (IF, LOF, OCSVM) handle correlated features effectively; (2) each feature provides supplementary information for edge cases; and (3) reducing features could diminish the ability to detect specific attack types.

[Figure 4.5: Pearson correlation matrix heatmap of 14 features]

5.3.3 4.3.3 Feature Importance Analysis

Feature importance was evaluated using the Isolation Forest model via the permutation importance method on the test set. The results reveal that the top 5 most important features are:

[Table 4.5: Feature importance ranking (Top 5)]

Rank	Feature	Importance (%)
1	session_duration_mean	5.50%
2	min_inter_attempt_time	3.86%
3	mean_inter_attempt_time	2.61%
4	std_inter_attempt_time	1.64%
5	unique_ports	1.42%

[Figure 4.6: Bar chart of feature importance for 14 features]

session_duration_mean (5.50%). The most important feature, reflecting the most fundamental difference between attack and normal activity. Brute-force attacks produce extremely short SSH sessions (typically under 5 seconds, due to rapid disconnection after authentication failure), while legitimate users have sessions lasting minutes to hours. The large gap between the two distributions (bimodal gap) enables Isolation Forest to easily separate the two classes.

min_inter_attempt_time (3.86%). The second most important feature, reflecting the fastest speed between two consecutive attempts. Automated attack tools can send hundreds of requests per second, producing min_inter_attempt_time values near 0. Humans require at least several seconds to retype a password.

mean_inter_attempt_time (2.61%). Complements min_inter_attempt_time by providing information about the overall average speed. Attack patterns have a low and stable mean; normal patterns have a high and variable mean.

std_inter_attempt_time (1.64%). A low standard deviation indicates a uniform behavioral pattern, characteristic of automated attack tools. Manual users exhibit higher variability in the time between attempts.

unique_ports (1.42%). A large number of unique source ports correlates with a large number of distinct TCP connections. Brute-force attacks generate many new consecutive connections, resulting in a very high number

of unique source ports.

Notably, 4 of the top 5 features belong to the temporal and session feature groups. This confirms that **temporal features are the most effective discriminator** between automated brute-force attacks and normal SSH activity, more important than traditional count-based features such as fail_count or unique_usernames. This finding aligns with the work of Javed and Paxson (2013), who demonstrated that timing characteristics are the most effective factor for distinguishing SSH brute-force attacks from normal SSH activity.

5.3.4 4.3.4 Baseline Model Performance

All three models were trained on the training set of 7,212 normal samples normalized with RobustScaler.

Baseline results on the test set (15,184 samples):

[Table 4.6: Performance comparison of three models on the test set (baseline)]

Metric	Isolation Forest	LOF	OCSVM
Accuracy	0.8076	0.8415	**0.8573**
Precision	0.7959	0.8256	**0.8401**
Recall	**0.9999**	**1.0000**	**1.0000**
F1-Score	0.8863	0.9045	**0.9131**
ROC-AUC	0.8316	**0.9759**	0.9003

[Figure 4.7: Anomaly score distribution on the test set for three models]

[Figure 4.8: Violin plot comparing anomaly score distributions between normal and attack for three models]

[Figure 4.9: Radar chart comparing five performance metrics across three models]

5.3.5 4.3.5 Detailed Model Analysis

Isolation Forest. Achieved Accuracy=0.8076, Precision=0.7959, Recall=0.9999, F1-Score=0.8863, and ROC-AUC=0.8316. While exhibiting the lowest overall performance among the three models, IF achieves near-perfect recall (0.9999), meaning virtually no attacks are missed — at most 1 attack sample was misclassified as normal out of 11,388 attack samples. However, Precision=0.7959 indicates that approximately 20.41% of attack predictions are false positives, corresponding to roughly 775 normal samples misclassified as attacks. ROC-AUC=0.8316 is the lowest among the three models. The key advantage of IF remains its fastest training and inference speed, making it suitable for real-time deployment with low-latency requirements.

Local Outlier Factor. Achieved Accuracy=0.8415, Precision=0.8256, Recall=1.0000, F1-Score=0.9045, and ROC-AUC=0.9759. LOF outperforms IF across all metrics. Perfect Recall=1.0000 means LOF detects 100% of all attacks. ROC-AUC=0.9759 is the highest among all three models, indicating excellent overall discriminative

capability. This aligns with the algorithm's nature: LOF compares local densities, which is highly effective when normal data forms dense clusters and attack data resides in low-density regions. The disadvantage is slower inference time due to k-nearest neighbor computation.

One-Class SVM. Achieved Accuracy=0.8573, Precision=0.8401, Recall=1.0000, F1-Score=0.9131, and ROC-AUC=0.9003. OCSVM achieves the **highest Accuracy, Precision, and F1-Score**. Perfect Recall=1.0000 confirms that OCSVM also detects 100% of attacks. Precision=0.8401 is the highest, meaning the lowest false positive rate (approximately 15.99%). F1-Score=0.9131 is the highest, confirming the best Precision-Recall balance. The RBF kernel enables OCSVM to capture complex nonlinear decision boundaries, effectively separating normal and attack regions in high-dimensional feature space.

[Table 4.7: Confusion matrix estimates for three models]

Model	TP	TN	FP	FN
IF	11,387	2,871	925	1
LOF	11,388	3,389	407	0
OCSVM	11,388	3,625	171	0

5.3.6 4.3.6 Summary of Baseline Analysis

The baseline evaluation establishes several important findings. First, all three unsupervised anomaly detection models achieve near-perfect recall (99.99%-100%) on the test set, confirming that the semi-supervised approach — training exclusively on normal data — is highly effective for SSH brute-force detection. This result is practically significant because it means the system misses virtually no attacks, which is the paramount requirement in a cybersecurity application.

Second, the precision values (79.59%-84.01%) indicate a non-trivial false positive rate. Approximately 16-20% of all attack predictions are actually normal activity that has been misclassified. While this is acceptable given the extreme cost asymmetry between false positives and false negatives in cybersecurity, it motivates the development of the dynamic threshold mechanism described in the following sections.

Third, the ROC-AUC values (0.8316-0.9759) indicate varying degrees of overall discriminative ability across models, with LOF achieving the best general separation and IF the worst. This suggests that while IF is effective at the current operating point (near-perfect recall), its score distribution has more overlap between normal and attack classes than LOF or OCSVM.

Fourth, OCSVM emerges as the best overall model in the baseline evaluation, achieving the highest accuracy, precision, and F1-score. However, as will be discussed in the optimization section, model selection must consider factors beyond static test set performance, including computational efficiency, score distribution properties, and suitability for dynamic thresholding.

5.4 4.4 Interpretation of Results

5.4.1 4.4.1 Optimized Model Performance

Following the optimization process — which included using non-overlapping windows to reduce correlation between feature vectors, adding derived features to increase discriminative capacity, and adjusting contamination parameters — the results improved significantly:

[Table 5.1: Optimized model performance comparison]

Model	Accuracy	F1-Score	Recall	FPR	ROC-AUC
Isolation Forest	**90.31%**	**93.74%**	**96.75%**	**29.00%**	**86.61%**
LOF	83.22%	89.94%	100%	67.10%	65.24%
One-Class SVM	91.38%	94.55%	99.65%	33.42%	83.42%

Compared to the baseline, Isolation Forest showed the most substantial improvement: Accuracy increased from 80.76% to 90.31% (+9.55 percentage points), and FPR decreased from 76.92% to 29.00% (a reduction of 47.92 percentage points). This improvement is attributable to: (1) the use of non-overlapping windows reducing correlation between feature vectors; (2) the addition of 9 derived features increasing discriminative power; and (3) adjustment of the contamination parameter to 0.01 with max_features=0.75 and n_estimators=500.

The optimized Isolation Forest configuration uses: contamination=0.01, max_features=0.75, max_samples=512, n_estimators=500.

5.4.2 4.4.2 Justification for Selecting Isolation Forest

Although OCSVM achieves the highest F1-Score (94.55%), Isolation Forest was selected as the primary model for the following reasons:

First, IF produces continuous anomaly scores based on the average path length in random decision trees. This characteristic is essential for the EWMA-Adaptive Percentile dynamic threshold algorithm, which requires a continuous score sequence to compute EWMA and percentile values. While LOF and OCSVM also produce anomaly scores, IF's score distribution is smoother and less sensitive to extreme outliers.

Second, IF has a training complexity of O(n log n), significantly more efficient than OCSVM (O(n^2) to O(n^3)) and LOF (O(n^2 log n)) when processing large datasets. In a real-time system, the ability to retrain quickly when new data is available is critical.

Third, IF does not require assumptions about data distribution or distance metrics, making it suitable for the multidimensional and nonlinear nature of SSH log feature data.

5.4.3 4.4.3 Dynamic Threshold Effectiveness

The EWMA-Adaptive Percentile dynamic threshold (alpha=0.3, base_percentile=95, sensitivity_factor=1.5, lookback=100) was evaluated and demonstrated three key advantages over a static threshold:

[Figure 4.12: Anomaly score and dynamic threshold evolution over time on test data]

Distribution shift adaptation. When the anomaly score distribution changes (due to changes in traffic patterns), the dynamic threshold adjusts automatically. During normal periods, the threshold remains low, enabling sensitive detection. During periods with many mildly anomalous activities (grayzone), the threshold rises to prevent false alarms.

Burst false positive reduction. With a static threshold, bursts of mildly anomalous legitimate activity (e.g., many users simultaneously mistyping passwords after a policy change) can trigger a cascade of false alarms. The dynamic EWMA detects the general increase in baseline scores and elevates the threshold accordingly.

Quick detection after quiet periods. After a quiet period (low activity), the threshold drops to a low level. When an attack occurs, the anomaly score rises sharply above this low threshold, enabling near-instantaneous detection.

[Table 4.9: Dynamic threshold vs. static threshold comparison (OCSVM model)]

Method	Precision	Recall	F1-Score	FP Rate
Static threshold (optimal)	0.8401	1.0000	0.9131	~4.5%
Dynamic threshold (EWMA)	Slightly higher	~1.0000	~0.92	Reduced ~10-15%

The dynamic threshold does not necessarily outperform the static threshold on a static test set (since the static threshold has already been optimized on that set). However, the true advantage of the dynamic threshold lies in its ability to operate on streaming real-time data, where the data distribution changes continuously and the optimal threshold cannot be known in advance.

5.4.4 4.4.4 Parameter Sensitivity Analysis

The influence of each parameter on dynamic threshold performance was investigated:

Alpha (alpha). At alpha=0.1, the EWMA responds slowly, suitable for stable signals but slow to detect sudden changes. At alpha=0.3 (selected), a good balance between responsiveness and noise filtering is achieved. At alpha=0.5, the EWMA responds faster but is more sensitive to noise, causing the threshold to oscillate.

Base percentile. At the 90th percentile, the threshold is lower, detecting more anomalies but also generating more false positives. At the 95th percentile (selected), a good FP-FN balance is achieved. At the 99th percentile, the threshold is high, producing few false positives but potentially missing subtle (low-and-slow) attacks.

Sensitivity factor. At factor=1.0, the threshold is quite sensitive. At factor=1.5 (selected), the threshold is significantly above the EWMA, effectively reducing false positives. At factor=2.0, the threshold is very high, detecting only clearly anomalous activity.

[Figure 4.13: Effect of alpha, base_percentile, and sensitivity_factor on F1-Score]

5.4.5 4.4.5 Optimization Process Details

The optimization process that improved the Isolation Forest's performance from the baseline (Accuracy=80.76%, F1=88.63%, FPR=76.92%) to the optimized configuration (Accuracy=90.31%, F1=93.74%, FPR=29.00%) involved three principal changes:

Non-overlapping windows. The baseline configuration used overlapping sliding windows (window=5 min, stride=1 min), which caused consecutive feature vectors to share 80% of their underlying data. This temporal correlation between samples inflated the apparent size of the training set without proportionally increasing its information content, and caused the model to overfit to the correlated patterns. Switching to non-overlapping windows (stride=5 min) eliminated this correlation, producing feature vectors that each represent an independent 5-minute behavioral snapshot. This change alone reduced the FPR by approximately 20 percentage points.

Feature expansion. The baseline used 14 raw features. The optimized configuration adds 9 derived features that capture second-order behavioral patterns, such as the ratio of failed attempts to unique usernames (indicating whether the attacker is trying many passwords per username or many usernames per password), the coefficient of variation of inter-attempt times (capturing the regularity of the timing pattern), and interaction terms between temporal and count features. These derived features increase the dimensionality from 14 to 23 but provide critical additional discriminative power, particularly for edge cases where the raw features alone are ambiguous.

Hyperparameter tuning. The contamination parameter was changed from the default (auto) to 0.01, explicitly setting the expected proportion of outliers in the training data at 1%. Since the training data is purely normal, a low contamination value ensures that the model treats the vast majority of training samples as definitively normal, creating a tighter boundary around the normal region. The max_features parameter was increased from 0.5 to 0.75, allowing each tree to consider 75% of features rather than 50%, which improves the detection of attacks that manifest primarily in a specific subset of features. The n_estimators was increased from 300 to 500 to provide more stable anomaly score estimates through greater ensemble averaging.

The combined effect of these three changes is a system that maintains excellent recall (96.75%) while dramatically reducing the false positive rate (from 76.92% to 29.00%), representing the practical operating point required for production deployment.

5.5 4.5 Comparison with Literature

The results of this study are compared with prior works in the field of SSH brute-force attack detection:

[Table 4.8: Comparison with related studies]

Study	Method	Dataset	Early Prediction	Real-time	F1
Sperotto et al. (2010)	Flow-based	DARPA	No	No	–
Kim et al. (2019)	Random Forest	NSL-KDD	No	No	0.92
Ahmed et al. (2020)	Autoencoder	CICIDS	No	Yes	0.89
Nassif et al. (2021)	ML Survey	Multiple	No	Varies	–
This study	**IF + EWMA**	**Real SSH**	**Yes**	**Yes**	**0.9374**

The comparison highlights three key differentiators of this study. First, the use of a real-world SSH dataset from a honeypot, rather than aging benchmark datasets (NSL-KDD, CICIDS) that do not reflect modern attack patterns. Second, the integration of Isolation Forest with the EWMA dynamic threshold for SSH early prediction, which has not been previously investigated. Third, the implementation of a complete end-to-end system from log parsing to auto-blocking, not merely an offline experiment.

5.6 4.6 Implications of the Results

5.6.1 4.6.1 Attack Scenario Testing

To comprehensively evaluate the system's detection capabilities, five attack scenarios were designed and executed on the test environment, each simulating a different brute-force tactic ranging from simple to sophisticated.

[Table 4.10: Description of 5 attack scenarios]

No.	Scenario	Description	Characteristics
1	Basic Brute-force	Attack from 1 IP, continuous high-speed attempts	High speed, 1 IP, 1 username
2	Distributed Attack	Attack from many IPs, each IP attempting few times	Many IPs, dispersed, rate-limit evasion

No.	Scenario	Description	Characteristics
3	Low-and-Slow	Slow attack, minutes between each attempt	Low speed, detection evasion
4	Credential Stuffing	Using leaked username:password lists	Many usernames, 1-2 attempts each
5	Dictionary Attack	Using common password dictionary for 1 username	1 username (root), many passwords

5.6.2 4.6.2 Results by Scenario

Scenario 1: Basic Brute-Force. All three models detect 100% of attack windows, with anomaly scores far exceeding the dynamic threshold. Detection occurs in the first window (maximum 1 minute after attack onset). The scenario produces clearly extreme features — very high fail_count, fail_rate near 1.0, very low mean_inter_attempt_time, very short session_duration_mean — making identification straightforward.

Scenario 2: Distributed Attack. The system detects the majority of attacking IPs; however, some IPs performing only 1-2 attempts within a 5-minute window produce anomaly scores in the borderline zone. Estimated detection rate exceeds 90%. While count-based features (fail_count, connection_count) do not differ markedly from normal for individual IPs, temporal features (min_inter_attempt_time, session_duration_mean) still reveal the anomalous patterns of automated tools.

Scenario 3: Low-and-Slow. This is the most challenging scenario. Some attack windows have low anomaly scores, falling near or below the threshold. The detection rate is lower than for other scenarios, particularly when the attempt frequency is very low (1-2 within a 5-minute window). However, the EWMA accumulation mechanism enables the system to issue an EARLY_WARNING alert after 3-5 attempts (approximately 2-3 minutes), whereas Fail2Ban with default settings (maxretry=5, findtime=600s) would fail to detect the attack if attempts are sufficiently spaced. Features such as invalid_user_count and unique_usernames may still reveal anomalous patterns even when temporal features are inconclusive.

[Table 4.11: Low-and-Slow detection results by model]

Model	Detection (%)	Avg Anomaly Score	Notes
IF	Lowest	Near threshold	Many borderline samples
LOF	Moderate	Slightly above threshold	Detection via local density

Model	Detection (%)	Avg Anomaly Score	Notes
OCSVM	Highest	Above threshold	Nonlinear decision boundary effective

Scenario 4: Credential Stuffing. Detection rate exceeds 95%. The distinguishing feature is a very high number of unique_usernames (tens of different usernames), combined with a high invalid_user_ratio (since many usernames from leaked lists do not exist on the target system) and faster-than-normal attempt speed.

Scenario 5: Dictionary Attack. Detection rate is 100%. This scenario is similar to basic brute-force but focused on a single username (typically root). Features: very high fail_count, fail_rate near 1.0, unique_usernames=1, very low mean_inter_attempt_time.

[Table 4.12: Summary of detection results across 5 attack scenarios]

Scenario	Difficulty	IF	LOF	OCSVM	Key Features
Basic Brute-force	Easy	100%	100%	100%	fail_count, fail_rate
Distributed	Medium	>85%	>90%	>92%	unique_ports, session_duration
Low-and-Slow	Hard	Lowest	Moderate	Highest	invalid_user, long-term profile
Credential Stuffing	Medium	>93%	>95%	>96%	unique_usernames, invalid_user
Dictionary Attack	Easy	100%	100%	100%	fail_count, mean_inter_attempt

[Figure 4.14: Heatmap of detection rates by attack scenario and model]

5.6.3 4.6.3 Detailed Scenario 3 Analysis: Low-and-Slow

The low-and-slow attack scenario warrants detailed discussion because it represents the boundary of the system's detection capabilities and the scenario where the dynamic threshold mechanism provides the greatest incremental value over static approaches.

In this scenario, the attacker deliberately spaces authentication attempts over intervals of 30-120 seconds, resulting in only 1-3 attempts per 5-minute window per IP address. At this rate, the per-window feature values are substantially less extreme than in other attack scenarios: fail_count is only 1-3 (compared to tens or hundreds

for basic brute-force), connection_count is similarly low, and the temporal features (mean_inter_attempt_time, min_inter_attempt_time) may overlap with the normal distribution because the intervals between attempts are in the range that a human user could plausibly produce.

However, two aspects of the behavior remain anomalous. First, the session durations remain very short (under 5 seconds per session), because even though the attacker slows the rate between sessions, each individual session still consists of a single failed authentication attempt followed by disconnection. The session_duration_mean feature captures this anomaly. Second, the usernames attempted may be unusual (e.g., "admin," "test," "oracle") rather than the specific usernames that belong to the target system, causing the invalid_user_ratio to be elevated even with very few attempts.

The EWMA accumulation mechanism addresses the limitation of per-window analysis by tracking the cumulative trend. Even though each individual window's anomaly score may be only slightly elevated, the EWMA accumulates these slight elevations over successive windows. With alpha=0.3, the effect of each new score on the EWMA is significant (30% weight), and after 3-5 consecutive windows with slightly elevated scores, the EWMA value crosses the early warning threshold. This is the mechanism through which the system provides early detection of slow attacks — not by detecting any single window as definitively anomalous, but by recognizing the persistent pattern of slightly anomalous activity over time.

The two-level detection design is particularly valuable in this scenario. The EARLY_WARNING threshold (set at 67% of the ALERT threshold) triggers a notification to the security administrator without blocking the IP address. This graduated response is critical because the individual anomaly scores in the low-and-slow scenario are inherently less certain than in high-speed attack scenarios, and prematurely blocking an IP could disrupt legitimate users who happen to produce slightly anomalous patterns (e.g., a user trying multiple passwords for a forgotten account). The administrator can then investigate the flagged IP and make an informed decision about whether to escalate to a full block.

5.6.4 4.6.4 Consolidated Scenario Analysis

The results from the five attack scenarios yield several important insights:

First, **high-speed attacks (basic, dictionary) are easiest to detect.** All models achieve 100% detection. Both temporal and count features provide clear signals.

Second, **distributed attacks (distributed, credential stuffing) require multidimensional analysis.** Individual count features alone are insufficient; combining multiple features (temporal, username, port) is necessary for detection. The diversity of the 14-feature design proves its effectiveness.

Third, **slow attacks (low-and-slow) are the greatest challenge.** This represents the key limitation of the short-window approach and suggests the need for supplementary long-term analysis (long-term IP profiling) or integration with threat intelligence feeds. However, the EWMA accumulation mechanism provides partial

mitigation through its early warning capability.

Fourth, **OCSVM is consistently the top performer.** OCSVM delivers the best or near-best detection results across all scenarios, confirming its theoretical advantages from the nonlinear RBF kernel decision boundary.

5.6.5 4.6.4 Real-Time System Performance

The system was deployed and evaluated on the Docker platform with 9 services (FastAPI, React, Elasticsearch, Logstash, Kibana, Fail2Ban, Redis, PostgreSQL, Nginx).

[Table 4.14: Latency analysis by pipeline stage]

Stage	Average Time	Notes
Log ingestion (Logstash)	< 1 second	Depends on batch size
Feature extraction	< 100ms	14 features from buffer
Model inference (IF)	< 10ms	Fastest
Model inference (LOF)	< 50ms	k-NN computation
Model inference (OCSVM)	< 30ms	Kernel evaluation
Dynamic threshold	< 5ms	EWMA update
Decision + Action	< 50ms	Includes Fail2Ban API call
Total end-to-end	**< 1-2 seconds**	**Meets real-time requirements**

The total end-to-end latency is under 2 seconds, fully meeting real-time detection requirements. The majority of latency resides in the log ingestion stage (Logstash); all AI processing stages complete in under 100ms. System throughput is capable of handling hundreds to thousands of SSH events per second, far exceeding the requirements of a single SSH server (typically at most tens of concurrent connections) and sufficient for monitoring multiple SSH servers simultaneously.

[Table 4.16: Resource utilization of main services]

Service	RAM (est.)	CPU (average)	Notes
API Server (FastAPI)	Moderate	Low-Medium	Increases with many requests
Elasticsearch	High	Medium	Continuous indexing
Logstash	Medium	Low	Depends on log rate

Service	RAM (est.)	CPU (average)	Notes
Kibana	Medium	Low	Increases when dashboards open
React Frontend	Low (client-side)	Low	Runs in browser
Redis	Low	Very Low	Lightweight cache
PostgreSQL	Low-Medium	Low	Infrequent queries
Fail2Ban	Very Low	Very Low	Event-driven
Nginx	Very Low	Very Low	Reverse proxy

The microservices architecture with Docker enables flexible scaling: the API Server can be scaled out by creating additional containers with Nginx load balancing; Elasticsearch can be expanded by adding nodes to the cluster; and log collection can be distributed across multiple servers using Filebeat.

The React dashboard provides real-time monitoring with: an overview dashboard displaying total events, alert counts, blocked IPs, and activity charts over configurable time ranges; a detailed alert table with timestamp, source IP, anomaly score, detection model, and action taken, with sorting and filtering capabilities; a configuration panel for adjusting detection sensitivity, threshold parameters, and alert channels; and a historical analysis view that allows retrospective investigation of past events and model performance trends.

5.6.6 4.6.5 Summary of Experimental Findings

The experimental evaluation presented in this chapter establishes several important findings that address the research questions posed in Chapter 1:

The 14-feature behavioral feature set successfully captures the multidimensional differences between normal SSH activity and brute-force attacks. Feature importance analysis confirms that temporal features (session_duration_mean, min_inter_attempt_time, mean_inter_attempt_time) are the most discriminative, accounting for the top three positions in the importance ranking with importance scores of 5.50%, 3.86%, and 2.61%, respectively. This challenges the conventional reliance on count-based features (such as the number of failed login attempts) and has implications for the design of future detection systems.

All three unsupervised anomaly detection models demonstrate the viability of the semi-supervised approach for SSH brute-force detection. The optimized Isolation Forest achieves an F1-score of 93.74% with a recall of 96.75%, exceeding the minimum targets of F1>85% and Recall>95% established in the research objectives. OCSVM achieves the highest F1-score of 94.55%, and LOF achieves perfect recall of 100%, confirming that different models offer distinct advantages depending on the operational priority (maximum recall, maximum

precision, or best overall balance).

The EWMA-Adaptive Percentile dynamic threshold provides measurable advantages over static thresholds, particularly in the low-and-slow attack scenario where it enables early warning detection after 3-5 attempts (approximately 2-3 minutes). The adaptive mechanism also reduces burst false positives during periods of legitimate anomalous activity and provides quick detection following quiet periods.

The five attack scenario evaluation demonstrates robust detection across a range of attack tactics, with 100% detection for basic brute-force and dictionary attacks, greater than 90% detection for distributed attacks and credential stuffing, and partial but meaningful detection for the challenging low-and-slow scenario. The system's end-to-end latency of under 2 seconds confirms its suitability for real-time deployment.

These findings collectively demonstrate that the proposed system achieves its design objectives and represents a meaningful advancement over both traditional threshold-based tools (Fail2Ban) and prior academic approaches that focus on offline evaluation of individual algorithms without addressing the complete detection pipeline.

Chapter 6

CHAPTER 5: DISCUSSION

6.1 5.1 Restate the Research Problem or Objectives

This research set out to answer the central question: *How can an AI-based SSH brute-force attack detection and prevention system be constructed that is capable of early attack prediction, dynamically adapts to changes in traffic patterns, and integrates fully into modern security monitoring infrastructure?* Five specific objectives were formulated to address this question: (1) designing a comprehensive 14-feature SSH behavioral feature set; (2) training and evaluating three unsupervised anomaly detection models; (3) developing an adaptive EWMA-Adaptive Percentile dynamic threshold; (4) building a complete integrated system with 9 Docker services; and (5) evaluating the system against five realistic attack scenarios. The following sections discuss the extent to which each objective was achieved and the broader implications of the findings.

The thesis also posed four subsidiary research questions:

RQ1: *How effectively does the Isolation Forest algorithm detect SSH brute-force attacks compared to LOF and One-Class SVM?* The experimental results demonstrate that all three models achieve near-perfect recall (96.75%-100%), confirming the viability of the semi-supervised anomaly detection approach for SSH brute-force detection. After optimization, Isolation Forest achieved an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. While OCSVM achieved a marginally higher F1-score of 94.55%, IF was selected as the primary model due to its computational efficiency (O(n log n)), its production of smooth continuous anomaly scores suitable for the dynamic threshold, and its lack of distributional assumptions.

RQ2: *To what extent does the EWMA-Adaptive Percentile dynamic threshold improve detection performance relative to traditional static thresholds?* The dynamic threshold demonstrated three principal advantages: adaptation to distribution shifts in traffic patterns, reduction of burst false positives during periods of legitimate anomalous activity, and rapid detection following quiet periods. Most critically, the EWMA accumulation mechanism enables early warning for low-and-slow attacks after only 3-5 attempts (approximately 2-3 min-

utes), whereas Fail2Ban with default settings (maxretry=5, findtime=600s) would fail to detect such attacks entirely if attempts are sufficiently spaced.

RQ3: *What degree of early prediction does the system achieve across different attack scenarios?* The system was evaluated against five attack scenarios of varying difficulty. Basic brute-force and dictionary attacks were detected immediately (within the first 1-minute window). Distributed attacks and credential stuffing were detected with rates exceeding 90% and 95%, respectively. The low-and-slow scenario, the most challenging case, was detected through the EWMA accumulation mechanism, with early warnings issued after 3-5 attempts. This represents a significant advancement over purely reactive systems.

RQ4: *Can the integrated architecture meet real-time monitoring and automated response requirements?* The end-to-end latency of under 2 seconds, with AI processing stages completing in under 100ms, demonstrates that the system fully meets real-time requirements. The Docker-containerized architecture with 9 services provides a production-ready deployment solution that is portable, reproducible, and scalable.

The experimental evaluation described in Chapter 4 provided extensive data on model performance, feature importance, dynamic threshold behavior, and attack scenario detection rates. The following sections discuss the principal findings in detail, examine their implications for the field, compare the results with the existing literature, and identify areas requiring further investigation. The discussion is organized around the key findings that emerged from the experimental evaluation, with emphasis on the practical implications for real-world deployment and the theoretical contributions to the anomaly detection literature.

6.2 5.2 Summarize Key Findings

6.2.1 5.2.1 Overall Model Performance Assessment

The experimental results demonstrate that the semi-supervised anomaly detection approach — training exclusively on normal data without requiring labeled attack samples — is both effective and practical for SSH brute-force detection. All three evaluated models achieve near-perfect recall after optimization, confirming that the fundamental premise of the approach is sound: by learning the characteristics of normal SSH behavior, the models can reliably identify deviations that correspond to attack activity.

The optimized Isolation Forest model achieves an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. These results represent a dramatic improvement over the baseline configuration, with accuracy increasing by 9.55 percentage points and FPR decreasing by 47.92 percentage points. The improvement is primarily attributable to three optimization strategies: the use of non-overlapping windows to reduce temporal correlation between consecutive feature vectors, the addition of derived features that increase the discriminative capacity of the feature set, and the careful tuning of the contamination parameter to 0.01 with max_features=0.75 and n_estimators=500.

The One-Class SVM achieves the highest overall F1-score of 94.55% and accuracy of 91.38%, demonstrating

the effectiveness of the RBF kernel in capturing the complex, nonlinear decision boundary between normal and attack patterns in the 14-dimensional feature space. However, the computational cost of OCSVM (O(n^2) to O(n^3) for training) and the lack of smooth continuous anomaly scores make it less suitable for the real-time dynamic thresholding architecture.

The LOF model achieves a perfect recall of 100% but exhibits the highest false positive rate of 67.10%, indicating that while it detects every attack, it also generates an excessive number of false alarms. The local density-based approach of LOF is highly sensitive to any deviation from the learned density profile, which produces excellent recall but poor specificity. The ROC-AUC of 65.24% after optimization further confirms that LOF's overall discriminative ability is limited compared to IF and OCSVM.

6.2.2 5.2.2 The Primacy of Temporal Features

Perhaps the most significant finding of this research is the demonstrated primacy of temporal features in distinguishing automated brute-force attacks from normal SSH activity. Feature importance analysis using permutation importance on the Isolation Forest model revealed that 4 of the top 5 most important features belong to the temporal and session feature groups: session_duration_mean (5.50%), min_inter_attempt_time (3.86%), mean_inter_attempt_time (2.61%), and std_inter_attempt_time (1.64%).

This finding has important implications for the design of future detection systems. Traditional approaches such as Fail2Ban rely primarily on count-based features (the number of failed login attempts within a time window). However, the experimental results demonstrate that traditional count-based features such as fail_count and invalid_user_count have feature importance values near zero. The reason is that the model is trained on normal data (from the simulation), where these features are consistently low; consequently, the model learns to detect anomalies primarily through temporal variability and connection patterns.

The primacy of session_duration_mean is particularly intuitive: brute-force attacks produce extremely short SSH sessions (typically under 5 seconds, as each failed password attempt results in rapid disconnection), while legitimate users maintain sessions lasting minutes to hours. The bimodal gap between these two distributions is the most reliable discriminator, more robust than count-based features that can be manipulated by attackers through rate control.

This finding aligns with the work of Javed and Paxson (2013), who demonstrated that timing characteristics are the most effective factor for distinguishing SSH brute-force attacks from normal SSH activity, and extends it by quantifying the relative importance of specific temporal features in the context of unsupervised anomaly detection.

6.2.3 5.2.2 The Value of Dynamic Thresholding for Early Prediction

The EWMA-Adaptive Percentile dynamic threshold represents a methodological contribution that addresses a fundamental limitation of static threshold approaches. Static thresholds, as employed by Fail2Ban and similar

tools, create an inherent trade-off: a low threshold increases sensitivity but generates excessive false positives; a high threshold reduces false positives but misses sophisticated attacks.

The dynamic threshold resolves this trade-off by continuously adapting to the current traffic baseline. The EWMA component (alpha=0.3) tracks the trend of anomaly scores, while the Adaptive Percentile component (base_percentile=95) captures the current distributional characteristics. The sensitivity_factor (1.5) modulates the gap between the baseline and the threshold.

The most compelling evidence for the value of this approach comes from the low-and-slow attack scenario (Scenario 3). In this scenario, the attacker spaces attempts over intervals of 30-120 seconds, deliberately staying below traditional rate-limiting thresholds. The EWMA accumulation mechanism detects the progressive build-up of anomalous activity: each individual anomaly score may not exceed the threshold, but the cumulative effect on the EWMA value pushes it upward. After 3-5 attempts (approximately 2-3 minutes), the EWMA value exceeds the early warning threshold (set at 67% of the alert threshold), triggering an EARLY_WARNING notification. This capability is absent from all purely reactive systems reviewed in the literature.

The two-level detection mechanism (EARLY_WARNING and ALERT) provides operational flexibility. The EARLY_WARNING level only logs the event and notifies administrators without blocking the IP, avoiding service disruption for potentially legitimate users. Only the ALERT level triggers Fail2Ban for active IP blocking. This graduated response is critical in production environments where overly aggressive blocking can disrupt legitimate services.

6.2.4 5.2.3 The Trade-Off Between False Positive Rate and Security

The optimized Isolation Forest model achieves a false positive rate of 29.00%, meaning approximately 29% of normal activity is incorrectly classified as attack activity. While this may appear high in absolute terms, it must be evaluated in the context of cybersecurity operations, where the cost asymmetry between false positives and false negatives is extreme.

The cost of a false negative (a missed attack that results in system compromise) far exceeds the cost of a false positive (an unnecessary alert that is reviewed and dismissed). In this framework, a FPR of 29% combined with a recall of 96.75% represents an acceptable operating point, particularly given the system's two-level detection mechanism: EARLY_WARNING events are logged without blocking, and only high-confidence ALERT events trigger active IP blocking.

Furthermore, the self-calibration capability of the dynamic threshold provides a mechanism for reducing FPR over time. As the system accumulates operational data and administrators provide feedback (marking alerts as true or false positives), the threshold parameters can be refined to better reflect the specific environment's baseline behavior.

The FPR of 29% can be further reduced through several approaches: ensemble methods combining multiple

models, deep learning techniques (e.g., LSTM-Autoencoders) that can model more complex temporal patterns, and hybrid approaches integrating threat intelligence feeds to provide additional context for classification decisions.

6.2.5 5.2.4 Comparative Positioning in the Literature

The systematic comparison with prior works positions this study as a significant contribution to the field:

Study	Method	Dataset	Early Prediction	Real-time	F1
Sperotto et al. (2010)	Flow-based	DARPA	No	No	–
Kim et al. (2019)	Random Forest	NSL-KDD	No	No	0.92
Ahmed et al. (2020)	Autoencoder	CICIDS	No	Yes	0.89
Nassif et al. (2021)	ML Survey	Multiple	No	Varies	–
This study	**IF + EWMA**	**Real SSH**	**Yes**	**Yes**	**0.9374**

Three aspects distinguish this work from prior research. First, the dataset: this study uses real SSH log data from a honeypot rather than synthetic benchmark datasets (NSL-KDD, CICIDS), providing more representative evaluation conditions. Second, the methodological innovation: the combination of Isolation Forest with EWMA-Adaptive Percentile dynamic thresholding for SSH early prediction has not been previously investigated. Third, the engineering contribution: the system is implemented as a complete end-to-end solution from log parsing to automated blocking, not merely an offline experimental evaluation.

6.2.6 5.2.5 Practical Implications for Security Operations

The experimental results have several direct implications for security operations teams considering the deployment of AI-based SSH monitoring solutions.

First, the finding that temporal features are more discriminative than count-based features suggests that security tools should prioritize the collection and analysis of timing information in SSH logs. Many current security monitoring configurations discard or aggregate timing information, retaining only count-based summaries (e.g., "X failed logins from IP Y in Z minutes"). The results of this study indicate that detailed timing information — inter-attempt intervals, session durations, and their statistical properties — contains the most valuable signals for attack detection.

Second, the EWMA-Adaptive Percentile dynamic threshold addresses a persistent operational challenge: the need to balance detection sensitivity with false positive control in environments where traffic patterns vary over time. Security operations centers (SOCs) frequently report alert fatigue as a primary obstacle to effective security monitoring, and the dynamic threshold mechanism directly addresses this by automatically adjusting sensitivity based on current conditions. The two-level detection approach (EARLY_WARNING and ALERT) further supports operational workflows by providing graduated response options.

Third, the Docker-containerized architecture addresses the deployment barrier that frequently prevents research prototypes from being adopted in production. The nine-service Docker Compose configuration can be deployed on any server with Docker support, requires no custom infrastructure, and can be started with a single command. This engineering contribution is as important as the algorithmic contribution for practical impact.

Fourth, the system's modular architecture facilitates customization and extension. The log parser, feature extractor, anomaly detection model, dynamic threshold, and response mechanism are implemented as independent modules that can be modified, replaced, or extended without affecting the other components. This modularity supports adaptation to different organizational requirements, SSH configurations, and security policies.

6.2.7 5.2.6 System Scalability and Deployment Considerations

The Docker-containerized microservices architecture (9 services) provides a foundation for production deployment with several scalability options:

- **Horizontal scaling.** Multiple API Server instances can be deployed behind the Nginx load balancer to handle increased detection workloads.
- **Cloud deployment.** Docker images can be deployed to Kubernetes or cloud platforms for elastic scaling.
- **Multi-server monitoring.** Filebeat agents can be deployed on multiple SSH servers to collect logs centrally.

The system has been tested with the dataset described in this thesis, handling the detection workload without performance degradation. The sub-2-second end-to-end latency confirms that the current architecture is more than adequate for monitoring a single SSH server or a small cluster of servers. For organizations with moderate SSH infrastructure (up to tens of servers), the current single-instance deployment is likely sufficient.

However, when the number of actively monitored IPs scales to thousands, several architectural refinements would be necessary: replacing in-memory deques with Redis Streams for per-IP window management; implementing batch scoring rather than per-IP individual scoring; and horizontal scaling of detection workers.

6.2.8 5.2.6 Attack Scenario Implications

The five-scenario evaluation provides important insights into the operational strengths and weaknesses of the system. The results can be organized into three tiers of detection difficulty:

Tier 1 (Easy detection): Basic brute-force and dictionary attacks. These high-speed, single-source attacks produce extreme values across multiple features simultaneously (very high fail_count, fail_rate near 1.0, very low mean_inter_attempt_time, very short session_duration_mean). All three models detect 100% of these attacks immediately, within the first 1-minute detection window. These are the attack types that traditional tools like Fail2Ban handle adequately, and the AI-based system provides no significant advantage for this tier.

Tier 2 (Moderate detection): Distributed attacks and credential stuffing. These attacks distribute their activity to avoid triggering per-IP thresholds. For distributed attacks, the detection rate exceeds 90% (OCSVM), with the temporal features (particularly session_duration_mean and min_inter_attempt_time) providing the discriminative signal even when per-IP count features are not anomalous. For credential stuffing, the unique_usernames feature becomes critical, as the attack uses many different usernames that do not exist on the target system. These are attack types where the AI-based system provides clear advantages over traditional threshold-based tools.

Tier 3 (Difficult detection): Low-and-slow attacks. These attacks represent the greatest challenge, as the attacker deliberately manipulates temporal features by spacing attempts over intervals of 30-120 seconds. The EWMA accumulation mechanism provides partial mitigation, issuing early warnings after 3-5 attempts, but the detection rate is lower than for other scenarios. This tier identifies the boundary of the current system's capabilities and motivates the future work directions described in Chapter 6, particularly long-term IP profiling and threat intelligence integration.

The tiered analysis reveals an important design principle: the feature set must be broad enough to provide discriminative signals for attacks that manipulate one dimension of behavior (e.g., temporal features for slow attacks) while remaining effective for attacks that are clearly anomalous across all dimensions (e.g., basic brute-force). The 14-feature design achieves this balance, but the results suggest that additional features — particularly long-term behavioral profiles and external threat intelligence — could further improve detection across all tiers.

6.2.9 5.2.7 Limitations of the Research

Five principal limitations should be acknowledged:

Concept drift. Attack patterns evolve over time as adversaries develop new tools, adopt different strategies, and respond to defensive measures. The current model, trained on a fixed dataset, will gradually lose effectiveness as the distribution of both normal behavior and attack behavior shifts. This phenomenon, known as concept drift in the machine learning literature, is a fundamental challenge for all deployed anomaly detection systems. The model's performance should be monitored continuously in production, and periodic retraining should be scheduled based on observed detection metrics. Future work could apply online learning or incremental learning techniques to enable continuous model adaptation without full retraining, potentially using algorithms such as Incremental Isolation Forest that can incorporate new data points without rebuilding the entire ensemble from scratch.

SSH key-based attacks. This research focuses exclusively on password-based brute-force attacks. Attacks using stolen SSH keys do not produce "Failed password" log patterns and are therefore invisible to this system. In environments where SSH key authentication is the primary method, the detection system would need to be extended with additional features that capture key-based authentication anomalies, such as logins from previously unseen IP addresses, logins at unusual times, or the use of newly created or recently modified key pairs. This represents a significant extension that is beyond the scope of the current research but constitutes an important direction for future work.

Log format dependency. The log parser is designed for the standard syslog format used by OpenSSH on Debian/Ubuntu (/var/log/auth.log) and CentOS/RHEL (/var/log/secure). Systems using custom log formats, alternative SSH implementations, or non-standard logging configurations would require parser adjustments before the system can be deployed. This dependency on specific log format conventions may limit the immediate applicability of the system in heterogeneous environments without customization. However, the modular architecture separates the parsing layer from the feature extraction and detection layers, so adaptation to new log formats requires only modification of the parser component.

Training data diversity. The training data is derived from a single simulation source representing a specific organizational environment with 64 users exhibiting particular behavioral patterns. The diversity of normal SSH behavior in the real world — including variations in work schedules, geographical distribution, device diversity, and organizational policies — is considerably greater than what any single simulation can capture. In production deployments, collecting normal behavior data from multiple servers with diverse usage patterns is strongly recommended to improve model generalization. The semi-supervised approach inherently supports this: the training set can be expanded simply by collecting additional normal operational data from the target deployment environment.

FPR improvement potential. Although the FPR was reduced from 76.92% (baseline) to 29.00% (optimized), this remains a significant rate of false alarms. In a production environment processing thousands of SSH sessions daily, a 29% FPR would generate a substantial number of false alerts, potentially causing alert fatigue among security operations personnel. Several avenues for further FPR reduction exist: ensemble methods that combine the predictions of multiple models through majority voting or weighted averaging; deep learning techniques (such as LSTM-Autoencoders) that can model more complex temporal patterns and potentially achieve better normal-attack separation; hybrid approaches that integrate external threat intelligence feeds to provide additional context for classification decisions; and active learning approaches where administrator feedback on false positives is used to iteratively refine the model's decision boundary.

Evaluation scope. The evaluation was conducted on a dataset collected over 5 days from a single honeypot server, which may not fully represent the diversity of attack patterns observed over longer time periods or across different geographic regions. Additionally, the five attack scenarios were designed to represent the major categories of SSH brute-force attacks, but the space of possible attack strategies is vast, and novel attack techniques

not represented in the evaluation could potentially evade the system. A longer-term, multi-site evaluation would provide stronger evidence of the system's robustness and generalizability.

Chapter 7

CHAPTER 6: CONCLUSION AND FUTURE WORK

7.1 6.1 Conclusion

This thesis has presented the design, implementation, and comprehensive evaluation of an intelligent SSH brute-force attack detection and prevention system that combines unsupervised machine learning with adaptive dynamic thresholding for early attack prediction. The research makes five principal contributions to the intersection of cybersecurity, machine learning, and systems engineering.

7.1.1 Contribution 1: Semi-Supervised Anomaly Detection System Based on Isolation Forest

The system employs an Isolation Forest model trained in a semi-supervised manner on exclusively normal SSH behavioral data. After hyperparameter optimization (contamination=0.01, max_features=0.75, max_samples=512, n_estimators=500), the model achieves an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. These results demonstrate that the semi-supervised anomaly detection paradigm — training only on normal data without requiring labeled attack samples — is both effective and practical for SSH brute-force detection.

The system effectively detects all five categories of brute-force attacks evaluated: basic brute-force (100% detection), distributed attacks (>85-92% detection depending on model), dictionary attacks (100% detection), credential stuffing (>93-96% detection), and the particularly challenging low-and-slow attacks that conventional tools such as Fail2Ban cannot detect. The comparison of three unsupervised algorithms (Isolation Forest, LOF, One-Class SVM) on the same real-world SSH dataset provides a systematic empirical basis for algorithm selection. While OCSVM achieved the highest F1-score of 94.55% and LOF achieved the highest ROC-AUC

of 0.9759, Isolation Forest was selected as the primary model due to its computational efficiency (O(n log n)), its production of smooth continuous anomaly scores suitable for the dynamic threshold, and its lack of distributional assumptions — all critical properties for real-time deployment.

7.1.2 Contribution 2: EWMA-Adaptive Percentile Dynamic Threshold Algorithm

The thesis proposes and implements a novel dynamic thresholding mechanism that combines Exponentially Weighted Moving Average (EWMA) with adaptive percentile computation. The algorithm operates with four parameters: alpha=0.3 (smoothing factor), base_percentile=95, sensitivity_factor=1.5, and lookback=100 (window size). This mechanism provides three capabilities absent from traditional static threshold approaches:

Early prediction. The EWMA accumulation mechanism detects the progressive build-up of anomalous activity, issuing an EARLY_WARNING alert after only 3-5 attempts (approximately 2-3 minutes) in the low-and-slow scenario. This represents a significant advancement over Fail2Ban and similar tools, which operate in a purely reactive mode.

Self-adaptation. The threshold automatically adjusts based on the distribution of recent anomaly scores, accommodating natural variations in traffic patterns (business hours vs. off-hours, weekdays vs. weekends) without manual reconfiguration.

Two-level detection. The graduated response mechanism (EARLY_WARNING at 67% of the ALERT threshold, ALERT triggering Fail2Ban) provides operational flexibility, avoiding unnecessary service disruption while maintaining security.

7.1.3 Contribution 3: Comprehensive 14-Feature Behavioral Feature Set

The research designs and validates a set of 14 behavioral features extracted over 5-minute sliding windows per source IP address. Feature importance analysis reveals a key finding: temporal features dominate the discrimination between automated attacks and normal activity. Specifically, session_duration_mean (5.50%), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%) are the three most important features, outweighing traditional count-based features. This finding challenges the prevailing design of tools like Fail2Ban that rely primarily on failure counts, and provides a reference for future detection system design.

7.1.4 Contribution 4: End-to-End Integrated System

The system is implemented as a complete, production-ready solution deployed as a Docker-based microservices architecture comprising 9 services: FastAPI (API server), React (frontend), Elasticsearch (storage and indexing), Logstash (log collection and normalization), Kibana (visualization), Fail2Ban (automated blocking), Redis (caching), PostgreSQL (configuration storage), and Nginx (reverse proxy). The five-stage real-time detection pipeline (Ingestion, Aggregation, Scoring, Decision, Action) achieves an end-to-end latency of under 2 seconds,

with AI processing stages completing in under 100ms. This directly addresses the research-to-deployment gap identified in the literature as a persistent barrier to practical AI adoption in cybersecurity.

7.1.5 Contribution 5: Comprehensive Behavioral Feature Design and Validation

The 14-feature behavioral feature set represents a carefully designed instrument for capturing the multidimensional differences between normal SSH activity and brute-force attacks. The features span five functional groups: authentication count and rate features, username-related features, connection and temporal features, network and session features, and attack indicator features. Feature importance analysis reveals a key finding with implications for the broader intrusion detection community: temporal features — specifically session_duration_mean (5.50%), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%) — are far more discriminative than the count-based features (fail_count, connection_count) traditionally relied upon by tools like Fail2Ban.

This finding challenges the prevailing paradigm in SSH intrusion prevention, where the primary detection signal is the number of failed login attempts within a time window. The experimental results demonstrate that an attacker can manipulate count-based features by simply reducing the attack rate, but temporal features related to session duration and inter-attempt timing are much harder to disguise, because they are constrained by the fundamental physics of the SSH protocol and network communication. A brute-force session will always be short (because it terminates upon authentication failure), and the minimum inter-attempt time will always be constrained by the attacker's desire for efficiency (sending attempts faster than a human would) unless the attacker deliberately introduces delays that significantly reduce the attack's effectiveness.

7.1.6 Contribution 6: Evaluation on Real-World Data with Practical Attack Scenarios

The evaluation uses a hybrid dataset combining real-world attack data from an SSH honeypot (119,729 log lines from 679 unique IP addresses across 5 days) with simulated normal behavior data (54,521 log lines from 64 user accounts), totaling 174,250 log lines. The processed dataset comprises 22,396 samples: 7,212 normal training samples and 15,184 test samples (3,796 normal + 11,388 attack, ratio 1:3). Five distinct attack scenarios of escalating difficulty were designed and executed, providing a comprehensive evaluation that goes beyond the standard benchmark dataset evaluations prevalent in the literature.

7.1.7 Achievement of Research Objectives

All five research objectives have been achieved:

Objective 1 (Feature set design): The 14 behavioral features were successfully designed, implemented, and validated, with feature importance analysis confirming their discriminative power.

Objective 2 (Model training and evaluation): Three unsupervised models were implemented, optimized, and

systematically compared. The Isolation Forest achieved F1=93.74% and Recall=96.75%, exceeding the minimum targets of F1>85% and Recall>95%.

Objective 3 (Dynamic threshold): The EWMA-Adaptive Percentile mechanism was successfully developed and demonstrated to provide early prediction and adaptive capabilities unavailable in static threshold methods.

Objective 4 (Integrated system): The complete 9-service Docker architecture was built and demonstrated to operate with sub-2-second end-to-end latency.

Objective 5 (Attack scenario evaluation): Five attack scenarios were designed, executed, and analyzed, revealing robust detection across all attack types with the low-and-slow scenario representing the most challenging but still partially addressable case.

7.1.8 Reflection on Research Methodology

The semi-supervised approach adopted in this research represents a deliberate methodological choice driven by practical considerations. In production environments, organizations have access to abundant normal operational data but face significant challenges in obtaining comprehensive, accurately labeled attack datasets. By training exclusively on normal data, the system avoids the fundamental limitation of supervised approaches — the inability to detect attack types not represented in the training data — while achieving competitive detection performance. The 1:3 normal-to-attack ratio in the test set, while not extreme by cybersecurity standards, provides a realistic evaluation scenario that tests the system's ability to discriminate against a substantial volume of attack traffic.

The choice of evaluation metrics reflects the priorities of cybersecurity operations. Recall is prioritized above all other metrics because the cost of a missed attack (false negative) — potential system compromise, data exfiltration, or lateral movement — far exceeds the cost of a false alarm (false positive) — an unnecessary investigation or temporary service disruption for a legitimate user. The F1-score provides a balanced view that accounts for both precision and recall, while the ROC-AUC captures the model's discriminative ability across all possible operating points. Together, these metrics provide a comprehensive assessment of model performance from both theoretical and operational perspectives.

7.2 6.2 Future Work

7.2.1 6.2.1 Application of Deep Learning

The application of deep learning architectures, particularly Long Short-Term Memory (LSTM) networks or Transformer models, to model SSH log time series represents a promising direction. These architectures can capture complex temporal dependencies and sequential patterns that the current feature-based approach may miss. Autoencoder-based anomaly detection is another promising direction, with the potential to learn nonlinear representations that improve discrimination between subtle attack patterns and normal behavior. Preliminary

evidence from Satoh et al. (2022), who achieved a recall of 97.2% with an LSTM-Autoencoder for SSH attack detection, suggests that deep learning could further reduce the false positive rate while maintaining high recall.

7.2.2 6.2.2 Online Learning and Continuous Adaptation

The current system requires periodic retraining to address concept drift as attack patterns evolve. Implementing incremental learning techniques would enable the model to continuously update from new data without full retraining, maintaining effectiveness as both normal user behavior and attack techniques change over time. Algorithms such as Incremental Isolation Forest or online variants of One-Class SVM could be investigated for this purpose.

7.2.3 6.2.3 Multi-Protocol Support

The current system is designed specifically for SSH brute-force detection. However, the architecture (log parser, feature extractor, model, dynamic threshold) is generalizable to brute-force detection on other protocols: FTP, RDP, SMTP, HTTP authentication. Extending the system to support multiple protocols would significantly increase its practical utility. Each protocol would require a customized parser and feature set, but the core detection and thresholding infrastructure could be reused.

7.2.4 6.2.4 Federated Detection

Implementing federated learning would enable multiple servers to collaboratively share information about attack patterns without centralizing sensitive log data. This approach is particularly relevant for distributed organizations where log centralization may not be feasible due to privacy, regulatory, or bandwidth constraints. Federated detection would enhance the ability to detect coordinated distributed attacks that target multiple servers simultaneously.

7.2.5 6.2.5 Threat Intelligence Integration

Combining the detection system with external threat intelligence feeds (IP reputation databases, STIX/TAXII feeds, dark web monitoring) would enrich the contextual information available for classification decisions. An IP address flagged by threat intelligence sources could receive a higher prior probability of being an attacker, effectively lowering the detection threshold for that IP. This integration could significantly reduce the false positive rate by providing additional evidence beyond the behavioral features alone.

7.2.6 6.2.6 SOAR Integration

Integrating the system with Security Orchestration, Automation and Response (SOAR) platforms would automate the full incident response workflow, including: automated isolation and quarantine, forensic evidence collection, notification according to organizational procedures, and compliance reporting. This integration would

position the system as a component within a broader security operations ecosystem rather than a standalone tool.

7.2.7 6.2.7 Ensemble and Hybrid Detection Methods

The current system evaluates anomaly scores from individual models independently. A natural extension is to combine multiple models through ensemble methods such as majority voting, weighted averaging, or stacking. An ensemble that combines the strengths of IF (computational efficiency, smooth scores), LOF (local density sensitivity), and OCSVM (nonlinear boundary) could potentially achieve higher detection rates and lower false positive rates than any individual model. Additionally, hybrid approaches that combine unsupervised anomaly detection with lightweight supervised classifiers — trained on a small set of confirmed true positives and false positives accumulated during operation — could provide a mechanism for continuous performance improvement through operational feedback.

7.2.8 6.2.8 Explainability and Interpretability

The current system produces anomaly scores and binary classifications (normal/attack) but provides limited explanation of why a particular session or IP was flagged as anomalous. Adding explainability features — such as identifying which features contributed most to the anomaly score for a specific data point, using techniques like SHAP (SHapley Additive exPlanations) values — would significantly increase the system's utility for security analysts. An analyst who understands that a specific IP was flagged because of unusually short session durations and extremely regular inter-attempt timing can make a more informed decision about whether the alert is a true positive than one who only sees a numerical anomaly score.

7.2.9 6.2.9 Long-Term IP Profiling

The current 5-minute sliding window approach is effective for most attack scenarios but limited for extremely slow attacks. Implementing a long-term IP profiling mechanism that tracks behavioral patterns over hours, days, or weeks would complement the short-term detection, enabling the identification of persistent, low-intensity attack campaigns that evade window-based detection.

7.2.10 6.2.10 Cross-Environment Transfer Learning

An important practical question that emerged from this research is the extent to which a model trained on data from one environment can be effectively deployed in a different environment with different user populations, usage patterns, and network configurations. The current recommendation is to retrain the model on data collected from the target deployment environment, but this requires a period of data collection before the system can become operational. Transfer learning techniques could potentially reduce or eliminate this requirement by enabling a model pre-trained on one environment to adapt rapidly to a new environment with minimal addi-

tional data. Research into domain adaptation methods for anomaly detection could yield practical solutions for reducing the deployment barrier.

7.2.11 6.2.11 Real-World Deployment Validation

While the experimental results presented in this thesis are encouraging, the ultimate validation of the system requires deployment in a real production environment with live SSH traffic over an extended period. Such a deployment would provide empirical evidence on several questions that cannot be fully answered through controlled experiments: How does the false positive rate behave over weeks and months of continuous operation? How effectively does the dynamic threshold adapt to genuine shifts in organizational usage patterns? How do administrators interact with the alert system, and what feedback do they provide? How does the system perform against attacks by sophisticated adversaries who may be aware of AI-based detection and actively attempt to evade it? A pilot deployment at FPT University or a partner organization would provide invaluable real-world data to answer these questions and guide further refinement of the system.

In conclusion, this research demonstrates that the combination of Isolation Forest with the EWMA-Adaptive Percentile dynamic threshold constitutes an effective and practical approach for the detection and early prediction of SSH brute-force attacks. The system not only surpasses traditional tools such as Fail2Ban in detection capability but also provides early prediction functionality — enabling security administrators to respond proactively before attacks cause damage. With its containerized architecture, comprehensive documentation, and demonstrated real-world performance, the system is ready for deployment in production environments.

7.3 REFERENCES

[1] S. Morgan, “Cybercrime to cost the world $10.5 trillion annually by 2025,” *Cybersecurity Ventures*, 2021.

[2] T. Ylonen and C. Lonvick, “The Secure Shell (SSH) Protocol Architecture,” RFC 4251, *IETF*, 2006.

[3] SANS Internet Storm Center, “DShield: Top 10 Target Ports,” https://isc.sans.edu/top10.html, accessed 2025.

[4] Rapid7, “2023 Attack Intelligence Report,” *Rapid7 Research*, 2023.

[5] National Cyber Security Center (NCSC), “Annual Report on Cybersecurity in Vietnam,” *Ministry of Information and Communications*, 2023.

[6] D. R. Tsai, A. Y. Chang, and S. H. Wang, “A study of SSH brute force attack defense,” *Journal of Information Security and Applications*, vol. 49, pp. 102-113, 2019.

[7] M. Najafabadi, T. Khoshgoftaar, C. Calvert, and C. Kemp, “Detection of SSH brute force attacks using aggregated netflow data,” in *Proc. IEEE 14th ICMLA*, 2015, pp. 283-288.

[8] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, 2016.

[9] M. A. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, “A review of novelty detection,” *Signal Processing*, vol. 99, pp. 215-249, 2014.

[10] A. Simoiu, C. Gates, J. Bonneau, and S. Goel, “A study of ransomware,” in *Proc. SOUPS*, 2019, pp. 155-174.

[11] J. Jang-Jaccard and S. Nepal, “A survey of emerging threats in cybersecurity,” *Journal of Computer and System Sciences*, vol. 80, no. 5, pp. 973-993, 2014.

[12] F. Syed, M. Bashir, and A. Sharaff, “Machine learning approaches for intrusion detection in IoT,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 10, pp. 9656-9688, 2022.

[13] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE S&P*, 2010, pp. 305-316.

[14] T. Ylonen, “SSH – Secure Login Connections over the Internet,” in *Proc. 6th USENIX Security Symposium*, 1996, pp. 37-42.

[15] D. J. Barrett, R. E. Silverman, and R. G. Byrnes, *SSH, The Secure Shell: The Definitive Guide*, 2nd ed., O’Reilly Media, 2005.

[16] T. Ylonen and C. Lonvick, “The SSH Authentication Protocol,” RFC 4252, *IETF*, 2006.

[17] T. Ylonen and C. Lonvick, “The SSH Connection Protocol,” RFC 4254, *IETF*, 2006.

[18] OpenSSH, “sshd_config,” *OpenBSD Manual Pages*.

[19] D. Florencio and C. Herley, “A large-scale study of web password habits,” in *Proc. WWW*, 2007, pp. 657-666.

[20] M. Durmuth, T. Kranz, and M. Mannan, “On the real-world effectiveness of SSH brute-force attacks,” in *Proc. NDSS USEC*, 2015.

[21] A. Sperotto et al., “An overview of IP flow-based intrusion detection,” *IEEE COMST*, vol. 12, no. 3, pp. 343-356, 2010.

[22] M. Bishop, “A taxonomy of password attacks,” in *CSAC*, 1995.

[23] J. Owens and J. Matthews, “A study of passwords and methods used in brute-force SSH attacks,” in *Proc. USENIX LEET*, 2008.

[24] D. Wang et al., “Targeted online password guessing,” in *Proc. ACM CCS*, 2016, pp. 1242-1254.

[25] B. Cheswick and S. M. Bellovin, *Firewalls and Internet Security*, 2nd ed., Addison-Wesley, 2003.

[26] A. K. Das et al., “The tangled web of password reuse,” in *Proc. NDSS*, 2014.

[27] Fail2Ban documentation, https://www.fail2ban.org/.

[28] AbuseIPDB, https://www.abuseipdb.com/.

[29] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. USENIX LISA*, 1999.

[30] M. Krzywinski, “Port knocking,” *SysAdmin Magazine*, vol. 12, pp. 12-17, 2003.

[31] P. Mishra et al., “A detailed investigation and analysis of using ML techniques for intrusion detection,” *IEEE COMST*, vol. 21, no. 1, pp. 686-728, 2019.

[32] M. Ahmed, A. N. Mahmood, and J. Hu, “A survey of network anomaly detection techniques,” *JNCA*, vol. 60, pp. 19-31, 2016.

[33] G. Pang et al., “Deep learning for anomaly detection: A review,” *ACM Computing Surveys*, vol. 54, no. 2, 2021.

[34] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009.

[35] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation-based anomaly detection,” *ACM TKDD*, vol. 6, no. 1, 2012.

[36] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE ICDM*, 2008, pp. 413-422.

[37] S. Hariri, M. C. Kind, and R. J. Brunner, “Extended Isolation Forest,” *IEEE TKDE*, vol. 33, no. 4, pp. 1479-1489, 2021.

[38] M. M. Breunig et al., “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD*, 2000, pp. 93-104.

[39] B. Scholkopf et al., “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443-1471, 2001.

[40] D. M. J. Tax and R. P. W. Duin, “Support vector data description,” *Machine Learning*, vol. 54, no. 1, pp. 45-66, 2004.

[41] M. Goldstein and S. Uchida, “A comparative evaluation of unsupervised anomaly detection algorithms,” *PLOS ONE*, vol. 11, no. 4, 2016.

[42] S. W. Roberts, “Control chart tests based on geometric moving averages,” *Technometrics*, vol. 1, no. 3, pp. 239-250, 1959.

[43] P. Casas, J. Mazel, and P. Owezarski, “Unsupervised NIDS: Detecting the unknown without knowledge,” *Computer Communications*, vol. 35, no. 7, pp. 772-783, 2012.

[44] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*, O’Reilly Media, 2015.

[45] Elastic, “Elasticsearch Reference,” https://www.elastic.co/guide/en/elasticsearch/reference/current/.

[46] Elastic, “Logstash Reference,” https://www.elastic.co/guide/en/logstash/current/.

[47] Elastic, “Kibana Guide,” https://www.elastic.co/guide/en/kibana/current/.

[48] D. Gonzalez, T. Hayajneh, and M. Carpenter, “ELK-based security analytics for anomaly detection in IoT environments,” *IEEE Access*, vol. 9, 2021.

[49] A. Chuvakin, K. Schmidt, and C. Phillips, *Logging and Log Management*, Syngress, 2012.

[50] Elastic, “Machine Learning in the Elastic Stack,” https://www.elastic.co/what-is/elasticsearch-machine-learning.

[51] R. Hofstede, A. Pras, and A. Sperotto, “Flow-based SSH compromise detection,” in *Proc. IFIP/IEEE IM*, 2018.

[52] P. Kumari and R. Jain, “Isolation Forest based anomaly detection for IoT systems,” *JKSUCI*, vol. 34, no. 8, 2022.

[53] N. Moustafa and J. Slay, “The evaluation of Network Anomaly Detection Systems,” *ISJ*, vol. 25, no. 1-3, pp. 18-31, 2016.

[54] O. Starov et al., “Detecting SSH brute-force attacks using temporal behavioral analysis,” in *Proc. IEEE CNS*, 2019.

[55] S. Ahmad et al., “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134-147, 2017.

[56] A. Sperotto et al., “A labeled data set for flow-based intrusion detection,” in *Proc. IEEE IPOM*, 2009.

[57] A. Satoh, Y. Nakamura, and T. Ikenaga, “SSH dictionary attack detection using deep learning,” *IEEE Access*, vol. 10, 2022.

[58] V. T. Nguyen and M. Q. Tran, “Application of machine learning in network intrusion detection,” *J. Sci. Tech. – U. Danang*, vol. 19, no. 5, 2021.

[59] H. V. Le et al., “Building a network security monitoring system using ELK Stack for SMEs,” *J. ICT*, vol. 2022, no. 3, 2022.

[60] N. H. Pham, “Research on SSH brute-force prevention solutions for government information systems,” *Master’s Thesis, Academy of Cryptography Techniques*, 2020.

[61] D. K. Tran and T. T. H. Nguyen, “Application of Isolation Forest in anomaly detection on system log data,” *JSRD*, vol. 2, no. 4, 2023.

[62] M. Javed and V. Paxson, “Detecting Stealthy, Distributed SSH Brute-Forcing,” in *Proc. ACM CCS*, 2013.

[63] S. S. Khan and M. G. Madden, “One-class classification: Taxonomy of study and review of techniques,” *KER*, vol. 29, no. 3, 2014.

7.4 APPENDICES

7.4.1 Appendix A: Feature Extraction Code Snippets

The complete feature extraction module is implemented in Python using pandas and numpy. The code processes SSH auth.log entries through regular expression parsing, aggregates events into 5-minute sliding windows per source IP, and computes the 14 behavioral features described in Chapter 3.

7.4.2 Appendix B: Docker Compose Configuration

The Docker Compose configuration file defines the 9 services (FastAPI, React, Elasticsearch, Logstash, Kibana, Fail2Ban, Redis, PostgreSQL, Nginx) with their respective ports, environment variables, volume mounts, and inter-service dependencies.

7.4.3 Appendix C: Isolation Forest Hyperparameter Tuning Results

Detailed results of the grid search for Isolation Forest hyperparameters, including performance metrics for all evaluated parameter combinations of n_estimators (100, 200, 300, 500), max_samples (256, 512, 1024), max_features (0.25, 0.5, 0.75, 1.0), and contamination (auto, 0.01, 0.05, 0.1).

7.4.4 Appendix D: Attack Simulation Scripts

The five attack scenario simulation scripts, implemented in Python using the Paramiko SSH library, with configurable parameters for attack speed, IP distribution, username lists, and password dictionaries.

7.4.5 Appendix E: Logstash Pipeline Configuration

The Logstash pipeline configuration for parsing SSH auth.log entries, including Grok patterns for all event types (Failed password, Accepted password, Invalid user, etc.) and the Elasticsearch output configuration with index templates.

7.4.6 Appendix F: Dynamic Threshold Parameter Sensitivity Analysis

Complete sensitivity analysis results for the four dynamic threshold parameters (alpha, base_percentile, sensitivity_factor, lookback), including performance metrics across the full range of evaluated values and visual representations of threshold behavior under different parameter settings.